Phoenix: Post-Quantum Verifiable Voting with Delay-Use Malicious-Ballot-Box Privacy

Anonymous

Abstract. E-voting systems have to satisfy challenging requirements: balancing the need for transparency while ensuring vote secrecy; resisting both classical and quantum adversaries. Specifically, no work to date attests to a scheme with provably strong privacy guarantees and post-quantum (PQ) settings simultaneously; privacy proofs either rely on weaker definitions or are absent entirely. In this work, we study the existing PQ voting schemes' structures, privacy weaknesses and ambiguities, e.g., Epoque (S&P21) and EVOLVE (CCS17). Then, we build Phoenix, the first PQ verifiable voting system where the ballot box cannot learn individual votes even when it is malicious and verification happens after tallying (du-mb-BPRIV). Moreover, we provide formal machine-checked security proofs of Phoenix's du-mb-BPRIV in EasyCrypt. While using Fully Homomorphic Signatures (FHS) in designing Phoenix, we identified a further gap in FHS: existing constructions either provide context-hiding but require heavy trapdoor samples per dataset, or achieve efficient evaluation without context-hiding, and no FHS offers both properties simultaneously. So, we design an FHS that is not only efficient for real-world applications, but also provides context-hiding. Eventually, we implement Phoenix via standard BFV encryption in Microsoft SEAL library and IBM Research's LaZer library for proof of knowledge and benchmark Phoenix on different CPUs.

1 Introduction

The idea of electronic voting is to modernize democratic processes through more accessibility and efficiency [1]; however, apart from the fact that people trust more in simple election systems with easily understandable procedures than complicated ones [2], designing voting schemes that can meet a wide range of security and privacy requirements remains one of the challenging problems in cryptography [3]. The paradox lies between transparency toward public verifiability and a high level of privacy for individual votes [2], as the level of privacy deceases by increasing verifiability and vice versa. This challenge gets more complicated by the advent of quantum computers, which will be in a position to retroactively break today's classically encrypted ballots [4].

Transition to post-quantum (PQ) e-voting needs to happen as soon as possible, because of the Store Now, Decrypt Later attack [5], but migration from classical cryptosystems to post-quantum ones is not trivial and all applications face many issues for this transition [6], from the substantial increase in key and ciphertext sizes imposed by post-quantum schemes [7] to interoperability and backward compatibility challenges arising during the coexistence of classical and quantum-resistant algorithms [8]. However, in voting systems, apart from these issues, this migration has even extra challenges, as mentioned earlier on balancing privacy and verifiability.

The current post-quantum electronic voting schemes need to go beyond innovations in building block and consider bigger picture by taking account formal cryptographic privacy and verifiability properties in voting, because elections are more than just counting ballots [1]. No matter how much novel and efficient a new building block for voting scheme is, it will not be securely-constructive if the resulting voting system can not satisfy cryptographic privacy definitions (e.g., ballot privacy [9]). While studying post-quantum voting schemes [4, 10–19], we observed that if a traditional voting system is ballot-private, it is not straightforward to instantiate a post-quantum variant of it [4], and just replacing its classical building blocks with post-quantum ones does not necessarily make the resulting voting system ballot-private. This is because, even when retaining the same architecture, migrating to post-quantum primitives (I) alters the underlying computational assumptions, thereby affecting information flow, privacy guarantees, and trust models [10]; and (II) introduces building blocks that, despite preserving classical properties (e.g., homomorphism), may possess additional attributes [4] capable of leaking information.

As summarized in Table 1, our study reveals a consistent pattern: no existing scheme simultaneously achieves strong ballot privacy, formal verifiability, and practical efficiency under post-quantum assumptions. Revoting support [20], important for usability and coercion mitigation, is similarly absent from all but one [15] scheme.

As part of our endeavor, we noticed the absence of a PQ primitive that simultaneously binds ballots to voter credentials, supports homomorphic tallying, and hides individual inputs from the public transcript. Classical approaches, mix-nets or homomorphic tallying with standard signatures, fail in the lattice setting, where signatures do not compose cleanly with homomorphic encryption and mix-nets [11] assume an honest server without malicious-mixing detection. FHS [21] offers an alternative, which lets a third party produce a publicly verifiable signature on the *output* of a computation over signed data. However, Gorbunov et al.'s context-hiding FHS [21] requires $O(N)$ trapdoor samples per N-element dataset, whereas Luo et al.'s efficient construction [22] eliminates this cost but lacks context-hiding. No existing FHS provides both, yet without context-hiding a derived tally signature may leak individual ballot information.

1.1 Contributions

To address the gaps identified above, in this work we:

- conduct a study of post-quantum voting schemes (Table 1), and identify limitations and ambiguities in the privacy proofs.
- construct the first efficient leveled fully homomorphic signature scheme with context-hiding, by building on the efficient evaluation of Luo et al. [22] while adding a context-hiding mechanism via the SampleLeft–SampleRight paradigm of Gorbunov et al. [21].
- build Phoenix, the first PQ verifiable voting scheme that achieves delay-use malicious-ballot-box privacy [23], together with strong correctness, strong consistency, ideal functionality [9], and strong verifiability (individual, universal, and eligibility) [24], all under standard lattice assumptions, along with supporting revoting. Also, we provide machine-checked privacy proof of Phoenix in EasyCrypt [25] tool-set framework.
- provide an implementation of Phoenix [26] and benchmark ballot creation across four platforms to demonstrate that end-to-end ballot creation completes in under 70 ms even for elections with up to 2^{25} candidates.

1.2 Related Works

The voting systems that are resistant to quantum attacks have diverse structures, where Table 1 summarizes our study of them.

Starting with Epoque [10], its privacy proof uses a weak *Vote Privacy* definition [27], which restricts the class of protocols, may yield inconsistent results under natural result functions (e.g., majority [28]), fails to apply to protocols outputting auxiliary data (e.g., tally proofs in ElectionGuard [29]), and misses privacy violations beyond same-vote detection [9]. Moreover, their Lemma 5 [30] omits the full hybrid argument for secret-share uniformity under commitment hiding. Also, the verifiability definition used [31] in Epoque does not distinguish ballot deletion from ballot modification [24]. Moreover, concrete post-quantum parameters for all building blocks are also absent in Epoque.

Hough et al. proposed NTRU Mix-Net [11], which NTRU parameters must be carefully chosen to avoid the overstretched regime where $q > 0.0058 \cdot \sigma^2 \cdot d^{2.484}$ (with d the polynomial degree, q the modulus, and σ the noise distribution width) enables subfield lattice attacks, and the scheme's privacy fundamentally relies on the assumption that at least one mix server remains honest without providing detection mechanisms for malicious mixing beyond proof verification. Although Hough et al. [11] does not propose a complete voting scheme, upon using their NTRU Mix-Net, they refer the privacy of resulting system to another mix-net by Aranha et al. [32], but even their theoretical system (full version's Appendix H [33]) does not provide any proofs or formal guarantees.

FOO-based Voting [4] combines threshold lattice-based blind signatures [34–36] with Dual-Regev encryption [37, 38], but one-more unforgeability of the underlying blind signature remains conjectural [39], and privacy breaks if the required perfectly anonymous submission channel is compromised.

Code-based Voting [12], the first code-based verifiable voting system, relies on the less-studied dual-LPN assumption whose cryptanalytic background is limited, concrete 128-bit parameters against ISD attacks remain uncertain [40], and ballot privacy claims lack formal proofs.

EVOLVE [13] modifies the BPRIV definition [9] but the adaptation sidesteps the simulator requirement by making the tally consistent with the board in both worlds; in standard BPRIV, $\beta = 1$ requires a simulator Sim to produce a fake proof π' consistent with BB_1, which EVOLVE never constructs, and whether their notion implies standard BPRIV is not formally analyzed. EVOLVE also proves only a weaker "realistic result" consistency rather than strong consistency [9]. EVOLVED [14] extends EVOLVE with Hamming Weight Proofs but inherits the same definitional gaps.

Chillotti et al. [15] present a homomorphic LWE-based voting scheme but provide only a proof sketch for ballot privacy. Abapour et al. [16] give a complete ballot privacy proof, but the scheme is exclusively for economic structures and, due to its blockchain dependency, lacks the flexibility for general use [41]; the same limitation applies to the blockchain-based schemes of Sekar et al. [17] and Mahanayak et al. [18] as well.

2 Preliminaries

Throughout this work, we use standard notation as λ denotes the security parameter; $[n]$ denotes the set $\{1,...,n\}$; $\|\cdot\|$ and $\|\cdot\|_\infty$ denote the Euclidean and infinity norms of a vector, respectively; $\widetilde{\mathbf{T}}$ denotes the Gram–Schmidt orthogonalization of a matrix $\mathbf{T}$.

Property/Scheme	Epoque [10]	NTRU-Mix [11]	FOO-based [4]	Code-based [12]	EVOLVE [13]	EVOLVED [14]	Homo-LWE [15]	BPRIV-BC [16]	Kyber-BC [17]	Hybrid-BC [18]	Stego-MQPC [19]	Phoenix [Ours]
Hard Assumptions	RLWE, RSIS	RLWE, NTRU	RLWE, RSIS	Dual-LPN	MLWE, MSIS	MLWE, MSIS	LWE	MLWE, MSIS	MLWE	ECDH*	MQ	RLWE, MLWE, SIS, MSIS
Building Blocks	IBE [42], Com [43], NIZK [44]	PKE [45], Com [46], ZKP [47–49]	BSign [34–36], TPKE [37,38]	Com [12], NIZK [12,50], PKE [51,52]	ZKP [53], Sign [54], Com [55], PKE [13]	ZKP [56], Com [13,56], HW-Proof [14], PKE [14]	FHE [57], Sign [58], NIZK [59]	PolyCom [60], PostSec [61], NIBSign [62]	KEM [63], Block [64,65], Hash [66]	ECDH [67], ABE [18], Block [68,69]	MQPC [70,71], LSB-Steg [19]	FHE [72], ZKP [73], FHS [21,22]
BPRIV [9]	Partial	×	×	×	✓	×	Partial	✓	×	×	×	✓
mb-BPRIV [74]	×	×	×	×	×	×	×	×	×	×	×	✓
dir-mb-BPRIV [23]	×	×	×	×	×	×	×	×	×	×	×	✓
Strong Consistency [9]	×	×	×	×	Partial	×	×	×	×	×	×	✓
Strong Correctness [9]	×	×	×	×	×	×	×	×	×	×	×	✓
Individual Verif. [24]	✓	×	×	✓	×	×	✓	Partial	Partial	Partial	×	✓
Universal Verif. [75]	✓	✓	Partial	✓	✓	Partial	✓	Partial	×	×	×	✓
Ideal Functionality [9]	×	×	×	×	×	×	×	×	×	×	×	✓
Revoting [20]	×	×	×	×	×	×	✓	×	×	×	×	✓

Table 1: Comparison of post-quantum electronic voting schemes based on cryptographic privacy properties with formal proofs providing and features they guarantee

2.1 Computational Assumptions

Let n,m,q be positive integers with q prime, $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{n\times m}$, and χ a distribution over $\mathbb{Z}_q$.

Short Integer Solution [76]. Given $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{n\times m}$, the $\mathsf{SIS}_{n,m,q,\beta}$ problem asks to find a non-zero vector $\mathbf{z} \in \mathbb{Z}^m$ such that $\mathbf{Az} = \mathbf{0} \pmod q$ and $\|\mathbf{z}\| \leq \beta$.

*Module-*SIS *[77].* Let $R_q = \mathbb{Z}_q[X]/(f(X))$ for an irreducible $f \in \mathbb{Z}[X]$ of degree n, and let $\mathbf{A} \leftarrow\$ R_q^{k\times m}$. The $\mathsf{MSIS}_{n,k,m,q,\beta}$ asks to find a non-zero $\mathbf{z} \in R_q^m$ such that $\mathbf{Az} = \mathbf{0}$ in R_q^k and $\|\mathbf{z}\| \leq \beta$, where $\|\cdot\|$ denotes the Euclidean norm of the coefficient vector.

Ring Learning With Errors [78]. Let $R_q = \mathbb{Z}_q[X]/(f(X))$. The $\mathsf{RLWE}_{n,q,\chi}$ *decision* problem asks to distinguish, with non-negligible advantage, the distribution $\{(a_i, a_i s + e_i)\}$ from $\{(a_i, u_i)\}$, where $a_i, u_i \leftarrow\$ R_q$, $s \leftarrow\$ R_q$, and $e_i \leftarrow\$ \chi$.

Module Learning With Errors [79]. Let $R_q = \mathbb{Z}_q[X]/(f(X))$, $\mathbf{A} \leftarrow\$ R_q^{k\times \ell}$, $\mathbf{s} \leftarrow\$ \chi^\ell$, and $\mathbf{e} \leftarrow\$ \chi^k$. The $\mathsf{MLWE}_{n,k,\ell,q,\chi}$ *decision* problem asks to distinguish $(\mathbf{A}, \mathbf{As} + \mathbf{e})$ from $(\mathbf{A}, \mathbf{u})$ with $\mathbf{u} \leftarrow\$ R_q^k$.

2.2 Building Blocks

We now state the cryptographic primitives that our Phoenix builds upon.

Definition 1 (Hash Function [80]). *A cryptographic hash function $H : \{0,1\}^* \to \{0,1\}^\lambda$ is a deterministic function that maps arbitrary-length inputs to fixed-length outputs of λ bits, satisfying:*

- *For any PPT adversary $\mathcal{A}$, $\Pr[(x,x') \leftarrow \mathcal{A}(1^\lambda) : x \neq x' \wedge H(x) = H(x')] \leq negl(\lambda)$ (Collision Resistance).*
- *For random $y \in \{0,1\}^\lambda$ and PPT adversary $\mathcal{A}$, $\Pr[x \leftarrow \mathcal{A}(y) : H(x) = y] \leq negl(\lambda)$ (Preimage Resistance).*

Definition 2 (BFV Encryption [72]). *The* BFV *scheme is a leveled homomorphic encryption scheme whose security relies on the hardness of the* RLWE *problem.* BFV = (KeyGen,Enc,Dec,Add,Rerand) *consists of:*

- $(pk,sk) \leftarrow \mathsf{KeyGen}(1^\lambda)$: *Outputs public encryption key pk and secret decryption sk.*
- $c \leftarrow \mathsf{Enc}(pk,\mu)$: *Encrypts plaintext μ under pk and outputs ciphertext c.*
- $\mu \leftarrow \mathsf{Dec}(sk,c)$: *Decrypts ciphertext c using sk and recovers plaintext μ.*
- $c \leftarrow \mathsf{Add}(c_1,c_2)$: *Homomorphically adds two ciphertexts, which satisfies the relation of* $\mathsf{Dec}(sk,\mathsf{Add}(c_1,c_2)) = \mathsf{Dec}(sk,c_1) + \mathsf{Dec}(sk,c_2)$.
- $c' \leftarrow \mathsf{Rerand}(pk,c)$: *Re-randomises c to produce a fresh ciphertext c' satisfying* $\mathsf{Dec}(sk,c') = \mathsf{Dec}(sk,c)$, *indistinguishable from a fresh encryption of same plaintext.*

Based on [81, 82], BFV satisfies: *correctness* ($\mathsf{Dec}(sk,\mathsf{Enc}(pk,\mu)) = \mu$ w.o.p. for all $\mu \in R_t$, provided noise stays below the decryption threshold); *IND-CPA security* under $\mathsf{RLWE}_{n,q,\chi_{\mathsf{err}}}$ ($|\Pr[\mathcal{A}(pk,\mathsf{Enc}(pk,\mu_0))=1] - \Pr[\mathcal{A}(pk,\mathsf{Enc}(pk,\mu_1))=1]| \leq \mathsf{negl}(\lambda)$ for all PPT $\mathcal{A}$, $\mu_0,\mu_1 \in R_t$); and BFV satisfies the *additive homomorphism* ($\mathsf{Dec}(sk,\mathsf{Add}(\mathsf{Enc}(pk,\mu_1),\mathsf{Enc}(pk,\mu_2))) = \mu_1 + \mu_2 \pmod t$).

Definition 3 (Lattice-Based Zero-Knowledge Proof System [73]). *The proof system* ZKP, *based on the hardness of* MSIS *and* MLWE, *proves knowledge of a witness satisfying a specified relation $\mathcal{R}$.* $\mathsf{ZKP} = (\mathsf{Setup},\mathsf{Prove},\mathsf{Verify})$ *consists of:*

- $pp \leftarrow \mathsf{Setup}(1^\lambda)$: *Outputs public parameters pp defining the relation $\mathcal{R}$.*
- $\pi \leftarrow \mathsf{Prove}(pp,x,w)$: *Given a statement x and witness w such that $(x,w) \in \mathcal{R}$, outputs a proof π via the Fiat–Shamir transform.*
- $\{0,1\} \leftarrow \mathsf{Verify}(pp,x,\pi)$: *Accepts if π is a valid proof that the prover knows a witness w such that $(x,w) \in \mathcal{R}$, and rejects otherwise.*

In Phoenix, proved relation is $\mathcal{R}_{\mathsf{vote}} = \{((c,pk),(v,\mathbf{s}_1,\mathbf{e},\mathbf{s}_2)) \mid c = \mathsf{BFV.Enc}(pk,v;\mathbf{s}_1,\mathbf{e},\mathbf{s}_2) \wedge v \in \{0,1\} \wedge \|\mathbf{s}_1\|,\|\mathbf{e}\|,\|\mathbf{s}_2\| \leq \beta_{\mathsf{enc}}\}$, where the witness is the secret bit v and the randomness-error triple $(\mathbf{s}_1,\mathbf{e},\mathbf{s}_2)$ bounded by the BFV noise parameter β_{enc}.

Via the Fiat–Shamir transform, all ZKPs are non-interactive (secure in ROM). Considering [73, 83–85], proof system satisfies: *completeness* (Verify accepts an honest proof for any valid witness $(\mathbf{s}_1,\mathbf{m},\mathbf{s}_2)$ w.o.p.); *knowledge soundness* under MSIS (any PPT adversary producing an accepting proof for a false statement yields, via the ABDLOP extractor [73], a short vector breaking MSIS with non-negligible probability); and *computational zero-knowledge* under MLWE (a PPT simulator produces transcripts computationally indistinguishable from real proofs, so π leaks nothing beyond the truth of the asserted relations).

3 Efficient Context-Hiding Leveled FHS

Existing FHS constructions face a gap: Gorbunov et al. [21] achieves context-hiding but requires $O(N)$ trapdoor samples per N-element dataset, while Luo et al. [22] is efficient but lacks context-hiding, which may allow a derived tally signature to leak individual ballot information; we resolve this by presenting the first FHS that achieves both properties simultaneously.

3.1 Syntax and Properties

Definition 4 (Fully Homomorphic Signature). *Let $\ell, d_{\max} \in \mathbb{N}$ denote the maximum dataset size, and maximum supported circuit depth, respectively, and let $\mathcal{C}$ be a class of boolean circuits of depth at most $d_{\max}$ over* NAND *gates. The* leveled fully homomorphic signature FHS *consists of PPT algorithms as:*

- $(pk,sk) \leftarrow \mathsf{Setup}(1^\lambda, 1^{|\tau|}, 1^\ell, 1^{d_{\max}})$: *Given security parameter and bounds on tag length, dataset size, circuit depth, outputs a public key pk and a signing trapdoor sk.*
- $vk \leftarrow \mathsf{KeyGen}(pk,\tau)$: *Given pk and a dataset tag* $\tau \in \{0,1\}^{|\tau|}$, *derives a per-dataset verification key vk.*
- $\sigma_i \leftarrow \mathsf{Sign}(sk,vk,\tau,\mu_i,i)$: *Given trapdoor sk, verification key vk, tag* τ, *message bit* $\mu_i \in \{0,1\}$, *index* $i \in [\ell]$, *outputs signature* σ_i *authenticating dataset's i-th entry.*
- $\sigma_f \leftarrow \mathsf{Eval}(vk,\tau,\boldsymbol{\mu},\boldsymbol{\sigma},f)$: *Given vk, tag* τ, *message vector* $\boldsymbol{\mu} \in \{0,1\}^\ell$, *base signatures* $\boldsymbol{\sigma}$, *and a circuit* $f \in \mathcal{C}$, *outputs an evaluated signature* σ_f *authenticating* $f(\boldsymbol{\mu})$.
- $\{0,1\} \leftarrow \mathsf{Verify}(vk,\tau,\sigma_f,f)$: *Accepts if* σ_f *is a valid signature on* $f(\boldsymbol{\mu})$ *under tag* τ, *and rejects otherwise.*
- $\tilde{\sigma}_f \leftarrow \mathsf{Hide}(vk,y,\sigma_f)$: *Given vk, output value* $y = f(\boldsymbol{\mu})$, *evaluated signature* σ_f, *re-randomizes* σ_f *into context-hiding signature* $\tilde{\sigma}_f$ *that reveals nothing beyond* y.
- $\{0,1\} \leftarrow \mathsf{HVerify}(vk,\tau,\tilde{\sigma}_f,y,f)$: *Accepts if* $\tilde{\sigma}_f$ *is a valid context-hiding signature certifying that* f *evaluated to* y *under tag* τ, *and rejects otherwise.*

Since our scheme operates on multiple datasets distinguished by tags τ, we adopt the multi-data syntax: a signer can sign arbitrarily many datasets under different tags, each tag τ producing a verification key vk via KeyGen, and the verifier must know the tag τ of the dataset on which the computation was performed.

Based on [21,22,86], we consider the following properties for FHS with a focus on unforgeability and context-hiding; for proofs of Theorems refer to Appendix A.

Definition 5 (Signing Correctness). *We say that the* FHS *scheme is* signing-correct *if for all tags* $\tau = \{\tau_0,\ldots,\tau_{|\tau|-1}\} \in \{0,1\}^{|\tau| \geq \lambda}$, *all messages* $\mu_i \in \{0,1\}$, *and all indices* $i \in [\ell]$,

$$\Pr\left[\begin{array}{l} (pk,sk) \leftarrow \mathsf{Setup}(1^\lambda,1^{|\tau|},1^\ell,1^{d_{\max}}); \\ vk \leftarrow \mathsf{KeyGen}(pk,\tau); \\ \sigma_i \leftarrow \mathsf{Sign}(sk,vk,\tau,\mu_i,i) \end{array} : \mathsf{Verify}(vk,\tau,\sigma_i,\mathsf{id}_i) = 1\right] = 1,$$

where id_i *denotes the projection circuit* $\mathsf{id}_i(\mu_1,\ldots,\mu_\ell) = \mu_i$ *and the probability is over the coins of the algorithms.*

Definition 6 (Evaluation Correctness). *We say that the* FHS *scheme is* evaluation-correct *if for all tags* $\tau \in \{0,1\}^{|\tau| \geq \lambda}$, *all message vectors* $\boldsymbol{\mu} = (\mu_1,\ldots,\mu_\ell) \in \{0,1\}^\ell$, *and all admissible circuits* $f \in \mathcal{C}$ *of depth at most* $d_{\max}$,

$$\Pr\left[\begin{array}{l} (pk,sk) \leftarrow \mathsf{Setup}(1^\lambda,1^{|\tau|},1^\ell,1^{d_{\max}}); \\ vk \leftarrow \mathsf{KeyGen}(pk,\tau); \\ \sigma_i \leftarrow \mathsf{Sign}(sk,vk,\tau,\mu_i,i) \text{ for } i \in [\ell]; \\ \sigma_f \leftarrow \mathsf{Eval}(vk,\tau,\boldsymbol{\mu},\boldsymbol{\sigma},f) \end{array} : \mathsf{Verify}(vk,\tau,\sigma_f,f) = 1\right] = 1,$$

where the probability is over the coins of the algorithms. Moreover, correctness extends to composed evaluation: for circuits $h_1,\ldots,h_s$ *with* $h_j : \{0,1\}^\ell \to \{0,1\}$ *and* $g : \{0,1\}^s \to \{0,1\}$, *if* (vk,σ_j) *satisfies* $\mathsf{Verify}(vk,\tau,\sigma_j,h_j) = 1$ *for each* $j \in [s]$, *then* $\sigma^* \leftarrow \mathsf{Eval}(vk,\tau,(h_1(\boldsymbol{\mu}),\ldots,h_s(\boldsymbol{\mu})),(\sigma_1,\ldots,\sigma_s),g)$ *satisfies* $\mathsf{Verify}(vk,\tau,\sigma^*,g \circ \bar{h}) = 1$, *where* $(g \circ \bar{h})(\boldsymbol{\mu}) = g(h_1(\boldsymbol{\mu}),\ldots,h_s(\boldsymbol{\mu}))$.

Definition 7 (Succinctness). *We say that the* FHS *scheme is* succinct *if for the security parameter* λ, *any tag* $\tau \in \{0,1\}^{|\tau| \geq \lambda}$, *all message vectors* $\boldsymbol{\mu} \in \{0,1\}^\ell$, *and all*

admissible circuits $f \in \mathcal{C}$,

$$\Pr\left[\begin{array}{l}(pk,sk) \leftarrow \mathsf{Setup}(1^\lambda,1^{|\tau|},1^\ell,1^{d_{\max}});\\ vk \leftarrow \mathsf{KeyGen}(pk,\tau);\\ \sigma_i \leftarrow \mathsf{Sign}(sk,vk,\tau,\mu_i,i) \text{ for } i \in [\ell];\\ \sigma_f \leftarrow \mathsf{Eval}(vk,\tau,\boldsymbol{\mu},\boldsymbol{\sigma},f)\end{array} : \|\sigma_f\| \le p(\lambda)\cdot\log\ell\right] = 1,$$

where $p(\lambda)$ *is a fixed polynomial in the security parameter. In particular, the size of the evaluated signature* σ_f *depends at most logarithmically on the dataset size* ℓ *and is independent of the circuit size of* f.

Definition 8 (Existential Unforgeability under Fully Adaptive Chosen-Message Attacks). *We define the security of the* FHS *scheme via the following experiment* $\mathsf{Expt}_{\mathcal{A}}^{\text{EU-F-ACMA}}(1^\lambda)$ *between a challenger and a PPT adversary* $\mathcal{A}$*:*

1. ***Setup.*** *The challenger runs* $(pk,sk) \leftarrow \mathsf{Setup}(1^\lambda,1^{|\tau|},1^\ell,1^{d_{\max}})$ *and sends* pk *to* $\mathcal{A}$.
2. ***Signing Queries.*** *The adversary* $\mathcal{A}$ *adaptively submits queries of the form* (τ,μ_i,i)*, where the tag* $\tau \in \{0,1\}^{|\tau| \ge \lambda}$ *is a dataset identifier,* $\mu_i \in \{0,1\}$*, and* $i \in [\ell]$*. The challenger maintains a list* T_τ *for each queried tag* τ *and responds with* $\sigma_i \leftarrow \mathsf{Sign}(sk,vk,\tau,\mu_i,i)$*, adding* (μ_i,i) *to* T_τ *(aborting if* $|T_\tau| > \ell$ *or if a different message was previously signed at the same index).*
3. ***Forgery.*** *The adversary* $\mathcal{A}$ *outputs a tuple* $(\tau^*,\sigma^*_{f^*},\mu^*,f^*)$*. The experiment outputs* 1 *if and only if* $\mathsf{Verify}(vk^*,\tau^*,\sigma^*_{f^*},f^*)=1$ *(or* $\mathsf{HVerify}(vk^*,\tau^*,\tilde{\sigma}^*_{f^*},\mu^*,f^*)=1$ *when context hiding is applied) and one of the following holds:*
   - Type I forgery: $\tau^* \neq \tau_j$ *for any previously queried tag* τ_j.
   - Type II forgery: $\tau^* = \tau_j$ *for some queried tag* τ_j*, and* $\mu^* \neq f^*(\boldsymbol{\mu}_j)$*, where* $\boldsymbol{\mu}_j$ *is the message vector signed under* τ_j.

We say that the FHS *scheme is* existentially unforgeable against fully adaptive chosen-message attacks *if for all PPT adversaries* $\mathcal{A}$*,* $\Pr[\mathsf{Expt}_{\mathcal{A}}^{\text{EU-F-ACMA}}(1^\lambda)=1] \le \mathsf{negl}(\lambda)$.

Definition 9 (Context-Hiding Security). *We say that the* FHS *scheme supports* statistical context hiding *if there exist additional PPT procedures* $\tilde{\sigma}_f \leftarrow \mathsf{Hide}(vk,y,\sigma_f)$ *and* $\mathsf{HVerify}(vk,\tau,\tilde{\sigma}_f,y,f)$ *such that the following three properties hold:*

1. ***Correctness of Hiding.*** *For any* $(pk,sk) \in \mathsf{Setup}(1^\lambda,1^{|\tau|},1^\ell,1^{d_{\max}})$*,* $vk \in \mathsf{KeyGen}(pk,\tau)$*, and any* (σ_f,y,f) *such that* $\mathsf{Verify}(vk,\tau,\sigma_f,f)=1$*, for any* $\tilde{\sigma}_f \in \mathsf{Hide}(vk,y,\sigma_f)$ *we have* $\mathsf{HVerify}(vk,\tau,\tilde{\sigma}_f,y,f)=1$.
2. ***Unforgeability.*** *The unforgeability guarantee of Definition 8 continues to hold when the* Verify *procedure in the security experiment is replaced by* HVerify.
3. ***Statistical Indistinguishability.*** *There exists a PPT simulator* Sim *such that for any fixed (worst-case) choice of* $(pk,sk) \in \mathsf{Setup}(1^\lambda,1^{|\tau|},1^\ell,1^{d_{\max}})$*,* $vk \in \mathsf{KeyGen}(pk,\tau)$*, and any* (σ_f,y,f) *satisfying* $\mathsf{Verify}(vk,\tau,\sigma_f,f)=1$*, we have*

$$\mathsf{Hide}(vk,y,\sigma_f) \overset{\text{stat}}{\approx} \mathsf{Sim}(sk,vk,\tau,y,f),$$

where the randomness is only over the coins of Hide *and* Sim*. In particular, the dataset tag component* τ *in the verification key* vk *depends only on* (sk,τ) *and not on the data* $\{\mu_i\}$.

3.2 Our Fully Homomorphic Signature Scheme

Definition 10 (Efficient Context-Hiding Leveled Fully Homomorphic Signature). *Let $\lambda,\ell,d_{\max}\in\mathbb{N}$ be the security parameter, maximum dataset size, and maximum circuit depth. Let n,m,q be lattice parameters with $m\geq 2n\lceil\log q\rceil$ determined by $(\lambda,d_{\max})$ under $\mathsf{SIS}_{n,m,q,\beta}$, $|\tau|\geq\lambda$ the tag bit-length, and $\tilde{\beta}_{\max}=\beta_{\max}\cdot p_{\mathrm{extra}}(n,m,\log q)$ the context-hiding norm bound. Built upon [22], the scheme uses:* lattice trapdoors $\mathsf{LT}=(\mathsf{GenTrap},\mathsf{SampleLeft},\mathsf{SampleRight})$ *[87, 88], where* $\mathsf{GenTrap}$ *outputs* $(\mathbf{A}^*,\mathbf{T}_{\mathbf{A}^*})$, $\mathsf{SampleLeft}$ *samples a short preimage under* $[\mathbf{A}^*\,|\,\mathbf{C}]$, *and* $\mathsf{SampleRight}$ *re-randomises a solution into a fresh statistically independent short preimage;* vector encoding $\mathsf{VE}=(\mathsf{encode},\mathsf{Unpack})$, *where* $\mathsf{encode}(k)$ *maps* $k\in\{0,1\}$ *to a structured offset matrix and* $\mathsf{Unpack}(\mathbf{R},i)$ *extracts the i-th block; and* tag evaluation $\mathsf{TE}=(\mathsf{PubEval},\mathsf{TrapEval})$, *where* $\mathsf{PubEval}_{(\tau,\nu,t)}$ *derives the per-tag matrix* $\mathbf{B}_\tau\in\mathbb{Z}_q^{n\times m}$ *from* $(\mathbf{B},\mathbf{B}')$. *The gadget matrix* $\mathbf{G}_n\in\mathbb{Z}_q^{n\times m}$ *[88] admits binary decomposition* $\mathbf{G}_n^{-1}(\cdot)$, $\mathbf{P}_{x_i}$ *is the permutation selecting the i-th gadget block, and the verification bound B is derived from* $s\geq\|\widetilde{\mathbf{T}_{\mathbf{A}^*}}\|\cdot\omega(\sqrt{\log m})$. *The scheme* $\mathsf{FHS}=(\mathsf{Setup},\mathsf{KeyGen},\mathsf{Sign},\mathsf{Eval},\mathsf{Verify},\mathsf{Hide},\mathsf{HVerify})$ *is given in Figure 1.*

$\mathsf{FHS.Setup}(1^\lambda,1^{|\tau|},1^\ell,1^{d_{\max}})$

$(\mathbf{A}^*,\mathbf{T}_{\mathbf{A}^*})\leftarrow\mathsf{GenTrap}(1^n,1^m,q)$
$\mathbf{A},\mathbf{D}_0,\mathbf{D}_1,\mathbf{B},\mathbf{B}'\leftarrow\$\,\mathbb{Z}_q^{n\times m}$
$\mathbf{r}_0\leftarrow\$\{0,1\}^m;\quad \mathbf{z}\leftarrow\mathbf{A}^*\cdot\mathbf{r}_0\in\mathbb{Z}_q^n$
$\nu\leftarrow\lceil\log\lceil\log|\tau|\rceil\rceil;\quad t\leftarrow\lceil 2\log 2|\tau|\rceil$
$pk\leftarrow(\mathbf{A},\mathbf{A}^*,\mathbf{D}_0,\mathbf{D}_1,\mathbf{B},\mathbf{B}',\mathbf{z},\nu,t)$
$sk\leftarrow\mathbf{T}_{\mathbf{A}^*}$
return (pk,sk)

$\mathsf{FHS.KeyGen}(pk,\tau)$

$\mathbf{B}_\tau\leftarrow\mathsf{PubEval}_{(\tau,\nu,t)}(\mathbf{B},\mathbf{B}',h_{\mathbf{z},\boldsymbol{\alpha},\boldsymbol{\beta}})$
$\mathbf{A}_\tau\leftarrow[\mathbf{A}^*\,|\,\mathbf{A}+\mathbf{B}_\tau]\in\mathbb{Z}_q^{n\times 2m}$
return $vk\leftarrow(\mathbf{A}_\tau,\mathbf{D}_0,\mathbf{D}_1,\mathbf{z})$

$\mathsf{FHS.Hide}(vk,y,\sigma_f)$

$(\mathbf{R}_f,y,f)\leftarrow\sigma_f$
Recompute $\mathbf{A}_\tau,\mathbf{D}_f$ from vk,f
$\mathbf{H}_f\leftarrow[\mathbf{A}_\tau\,|\,\mathbf{D}_f+(y-1)\mathbf{G}_n]\in\mathbb{Z}_q^{n\times 3m}$
return $\tilde{\sigma}_f\leftarrow\mathsf{SampleRight}(\mathbf{H}_f,\mathbf{R}_f,\mathbf{z})$

$\mathsf{FHS.Sign}(sk,vk,\tau,\mu_i,i)$

for $k\in\{0,1\}$ **do**
$\begin{bmatrix}\mathbf{R}_{k,1}\\ \mathbf{R}_{k,2}\end{bmatrix}\leftarrow\mathsf{SampleLeft}(\mathbf{A}^*,\mathbf{A}+\mathbf{B}_\tau,\mathbf{T}_{\mathbf{A}^*},\mathbf{D}_k+\mathsf{encode}(k),s)$
$\begin{bmatrix}\mathbf{R}_{\mu_i,1,i}\\ \mathbf{R}_{\mu_i,2,i}\end{bmatrix}\leftarrow\mathsf{Unpack}\left(\begin{bmatrix}\mathbf{R}_{\mu_i,1}\\ \mathbf{R}_{\mu_i,2}\end{bmatrix},i\right)$
return $\sigma_i\leftarrow\left(\begin{bmatrix}\mathbf{R}_{\mu_i,1,i}\\ \mathbf{R}_{\mu_i,2,i}\end{bmatrix},\mu_i,i\right)$

$\mathsf{FHS.Eval}(vk,\tau,\boldsymbol{\mu},\boldsymbol{\sigma},f)$

for each NAND-gate $g=(u,v,w)$ in f **do**
$\hat{\mathbf{D}}_u\leftarrow\mathbf{G}_n^{-1}(\mathbf{D}_u\mathbf{G}_n^{-1}(\mathbf{P}_{x_i}\mathbf{G}_n))$
$\mathbf{R}_w\leftarrow\mathbf{R}_v\hat{\mathbf{D}}_u-y\mathbf{R}_u$
$\mathbf{D}_w\leftarrow\mathbf{D}_v\mathbf{G}_n^{-1}(\mathbf{P}_{y_j}\mathbf{G}_n)\hat{\mathbf{D}}_u-\mathbf{G}_n$
return $\sigma_f\leftarrow(\mathbf{R}_f,f(\boldsymbol{\mu}),f)$

$\mathsf{FHS.Verify}(vk,\tau,\sigma_f,f)$

$(\mathbf{R}_f,y,f)\leftarrow\sigma_f$
Recompute $\mathbf{A}_\tau,\mathbf{D}_f$ from vk,τ,f
$e_1\leftarrow\|\mathbf{R}_f\|_\infty\leq B$
$e_2\leftarrow(\mathbf{A}_\tau\mathbf{R}_f=\mathbf{D}_f+y\mathbf{G}_n)$
return $(e_1\wedge e_2)$

$\mathsf{FHS.HVerify}(vk,\tau,\tilde{\sigma}_f,y,f)$

Recompute $\mathbf{A}_\tau,\mathbf{D}_f$ from vk,τ,f
$\mathbf{H}_f\leftarrow[\mathbf{A}_\tau\,|\,\mathbf{D}_f+(y-1)\mathbf{G}_n]$
$e_1\leftarrow(\mathbf{H}_f\cdot\tilde{\sigma}_f=\mathbf{z})$
$e_2\leftarrow(\|\tilde{\sigma}_f\|_\infty\leq\tilde{\beta}_{\max})$
return $(e_1\wedge e_2)$

Fig. 1: Our context-hiding leveled FHS, where a simulator $\tilde{\sigma}_f\leftarrow\mathsf{Sim}\ (sk,\ vk,\ \tau,\ y,\ f)$ produces output indistinguishable from Hide by the SampleLeft-SampleRight statistical closeness (see Lemma 6.4 of [21]). The matrix $\mathbf{G}_n$ denotes the gadget matrix and $B,\tilde{\beta}_{\max}$ the verification and context-hiding norm bounds, respectively.

3.3 Security Analysis of Our FHS

Theorem 1. *The* FHS *scheme of Definition 10 satisfies signing correctness (Definition 5).*

Theorem 2. *The* FHS *scheme of Definition 10 satisfies evaluation correctness (Definition 6).*

Theorem 3. *The* FHS *scheme of Definition 10 satisfies succinctness (Definition 7).*

Theorem 4. *Under* $\mathsf{SIS}(n,m,q,\beta)$*, for every PPT adversary* $\mathcal{A}$*,* $\Pr\big[\mathsf{Expt}^{\text{EU-F-ACMA}}_{\mathcal{A},\mathsf{FHS}}(1^\lambda)=1\big] \leq \mathsf{Adv}^{\mathsf{sis}}_{n,m,q,\beta}(\lambda) + \mathsf{negl}(\lambda)$*, where* FHS *is the scheme of Definition 10 and* $\mathsf{Expt}^{\text{EU-F-ACMA}}$ *is the experiment of Definition 8.*

Any winning adversary must produce $(\tau^*,\sigma^*_{f^*},\mu^*,f^*)$ with $\mathsf{Verify}(vk^*,\tau^*,\sigma^*_{f^*},f^*)=1$. In a Type I forgery, the extracted $\mathbf{R}^*_f$ satisfies $\mathbf{A}_{\tau^*}\mathbf{R}^*_f=\mathbf{D}_{f^*}+\mu^*\mathbf{G}_n$ with $\|\mathbf{R}^*_f\|_\infty \leq B$ on a never-queried tag, yielding a short SIS solution directly. In a Type II forgery, subtracting the verification equations for two valid signatures on the same tag τ_j but distinct outputs produces a non-trivial short kernel vector, again breaking $\mathsf{SIS}_{n,m,q,\beta}$.

Theorem 5. *Let* $(pk,sk)\leftarrow\mathsf{Setup}(1^\lambda,1^{|\tau|},1^\ell,1^{d_{\max}})$*,* $vk\leftarrow\mathsf{KeyGen}(pk,\tau)$*, and suppose* $\sigma_f=(\mathbf{R}_f,y,f)$ *satisfies* $\mathsf{Verify}(vk,\tau,\sigma_f,f)=1$*. Then for any* $\tilde{\sigma}_f\leftarrow\mathsf{Hide}(vk,y,\sigma_f)$*, we have* $\mathsf{HVerify}(vk,\tau,\tilde{\sigma}_f,y,f)=1$*.*

Theorem 6. *Under the* $\mathsf{SIS}(n,m,q,\beta)$ *assumption with* $\beta_{\mathsf{SIS}}=2^{\omega(\log\lambda)}\cdot\beta^2_{\max}$*, the* FHS *scheme remains existentially unforgeable when the verification procedure in Definition 8 is replaced by* HVerify*.*

Theorem 7. *The* FHS *satisfies Definition 9: for every (unbounded) adversary* $\mathcal{A}$*, every* $(pk,sk)\leftarrow\mathsf{Setup}(1^\lambda)$*,* $vk\leftarrow\mathsf{KeyGen}(pk,\tau)$*, and every* (σ_f,y,f) *with* $\mathsf{Verify}(vk,\tau,\sigma_f,f)=1$*,* $\Delta\big(\mathsf{Hide}(vk,y,\sigma_f),\mathsf{Sim}(sk,vk,\tau,y,f)\big)\leq\mathsf{negl}(\lambda)$*, where* $\Delta(\cdot,\cdot)$ *denotes statistical distance, following from the* SampleLeft–SampleRight *statistical closeness of [21].*

For any valid $\sigma_f=(\mathbf{R}_f,y,f)$, Hide calls $\mathsf{SampleRight}(\mathbf{H}_f,\mathbf{R}_f,\mathbf{z})$ with $\mathbf{H}_f=[\mathbf{A}_\tau \mid \mathbf{D}_f+(y-1)\mathbf{G}_n]$, while $\mathsf{Sim}(sk,vk,\tau,y,f)$ recomputes the same $\mathbf{H}_f$ and calls SampleLeft from the trapdoor without accessing $\mathbf{R}_f$. The SampleLeft–SampleRight statistical closeness of [21] implies $\Delta(\mathsf{Hide}(vk,y,\sigma_f),\mathsf{Sim}(sk,vk,\tau,y,f))\leq\mathsf{negl}(\lambda)$, so $\tilde{\sigma}_f$ reveals nothing beyond y.

3.4 Efficiency Comparison with Gurbanov et al. [21]

Here, we discuss how our scheme's time and space cost is more efficient than Gurbanov et al. [21], even by adding context-hiding property.

Signing complexity. In [21], Sign invokes SamPre once per data element $i\in[N]$, costing $O(N\cdot m^2n)$ time and $O(N\cdot m^2\log q)$ bits of storage. Our FHS calls SampleLeft exactly *twice* (for $k\in\{0,1\}$), regardless of ℓ; each per-message signature is then derived via the deterministic multiplication $[\mathbf{R}^\top_{\mu_i,1,i}\,|\,\mathbf{R}^\top_{\mu_i,2,i}]^\top=[\mathbf{R}^\top_{k,1}\,|\,\mathbf{R}^\top_{k,2}]^\top\cdot\mathbf{G}_n^{-1}(\mathbf{P}_i\mathbf{G}_n)$ at $O(m^2)$ per message (since $\mathbf{G}_n^{-1}(\mathbf{P}_i\mathbf{G}_n)\in\{0,1\}^{m\times m}$), giving total cost $O((\ell+n)\cdot m^2)$—a factor-$\Theta(n)$ improvement for $\ell\gg1$.

Context-hiding overhead and space cost. Both schemes achieve statistical context hiding via a single SampleRight/SampleLeft call, adding $\widetilde{O}(m^2n)$ overhead. Our hiding matrix $\mathbf{H}_f=[\mathbf{A}_\tau \mid \mathbf{D}_f+(y-1)\mathbf{G}_n]\in\mathbb{Z}_q^{n\times 3m}$ yields $\tilde{\sigma}_f\in\mathbb{Z}^{3m}$ versus $\tilde{\sigma}\in\mathbb{Z}^{2m}$ in [21]—a modest constant-factor increase. For public-key size, [21] requires $O(N\cdot nm\log q)$ bits for $(\mathbf{V}_1,\ldots,\mathbf{V}_N)$, while our $pk=(\mathbf{A},\mathbf{A}^*,\mathbf{D}_0,\mathbf{D}_1,\mathbf{B},\mathbf{B}',\mathbf{z})$ occupies only $O(nm\log q)$ bits

independent of ℓ, with $\mathbf{z} \in \mathbb{Z}_q^n$ adding a negligible $O(n\log q)$ bits. Secret keys are identical in both ($O(m^2\log q)$). Overall, our FHS achieves two pre-image samples instead of ℓ, dataset-independent public keys, and context hiding—at the cost of only one extra element $\mathbf{z}$ and a hidden signature of size $3m$ versus $2m$.

4 Post-Quantum Verifiable du-mb-BPRIV Voting System

Entities. Our voting system, Phoenix, involves four distinct roles. A registrar generates and privately distributes signing keys to voters, sending the corresponding verification keys to the server. Voters encrypt and sign their ballots, submitting them through an authenticated channel, with only their latest ballot retained if they re-vote. The voting server maintains the public bulletin board, validating and recording all accepted ballots. Finally, a group of decryption trustees collectively hold the election's private key under a threshold scheme, so that only a required minimum number of them need to participate to decrypt the final result.

Voting Protocol. In Phoenix, voters encrypt their vote under the election public key, prove its well-formedness via a ZKP, and sign the resulting ballot with their credential key, after which the server validates and records it on the BB for homomorphic tallying. A *revote policy* Policy is a function that, given a list of ballots, selects which ballot to count per voter. Two common instantiations are: (I) only the first valid ballot cast by each voter is retained, (II) only the last valid ballot cast by each voter is retained (which our system focuses on it). The Phoenix consists of the following procedures.

- Setup(1^λ): Runs $(pk,sk) \leftarrow \mathsf{BFV.KeyGen}(1^\lambda)$ and $pp \leftarrow \mathsf{ZKP.Setup}(1^\lambda)$. Returns the election key pair (pk,sk) and public parameters pp.
- Register(id): Runs $(pk_{fhs},sk_{fhs}) \leftarrow \mathsf{FHS.Setup}(1^\lambda,1^{|\tau|},1^\ell,1^{d_{\max}})$ and the $upk \leftarrow \mathsf{FHS.KeyGen}(pk_{fhs},id)$. Sets $usk = sk_{fhs}$. Returns the voter credential pair (upk,usk), where usk is the secret credential and upk is the corresponding public credential (recoverable from usk).
- Vote(id,v,pk,usk): For voter id with credentials (upk,usk), constructs a ballot as follows. First, encrypts the vote as $c \leftarrow \mathsf{BFV.Enc}(pk,v)$ to protect vote privacy. Then, produces a zero-knowledge proof $\sigma \leftarrow \mathsf{ZKP.Prove}(pp,c,v)$ to attest that c is a well-formed encryption of a valid vote, without revealing v itself. Finally, computes a signature $s \leftarrow \mathsf{FHS.Sign}(usk,upk,id,(c,\sigma),1)$ to bind the ballot to the voter's credential and prevent impersonation. Returns the ballot $b = (id,upk,c,\sigma,s)$.
- Valid(BB,b,pk): Parses $b = (id,upk,c,\sigma,s)$ and verifies three conditions: (1) no ballot in BB pairs id with a different credential, or uses upk for a different identity (i.e., each identity maps to exactly one credential, preventing double voting under different keys); (2) the zero-knowledge proof is valid: $\mathsf{ZKP.Verify}(pp,c,\sigma) = 1$, ensuring the encrypted vote is well-formed; (3) the signature is valid: $\mathsf{FHS.Verify}(upk,id,(c,\sigma),s) = 1$, confirming the ballot was created by the legitimate credential holder. Returns true iff all three conditions hold.

- Publish(BB): Applies the revote policy to the ballot box to obtain the set of relevant ballots $C = \mathsf{Policy}(BB)$, then publishes each ballot together with its hash fingerprint: $pbb = \{(b, H(b)) \mid b \in C\}$. Voter identities are stripped from published ballots to protect anonymity.
- Tally(BB,sk): Applies Policy to BB to select one ballot per voter, discards any ballots failing the proof or signature checks, and collects the remaining ciphertexts $\{c_i\}$. Exploiting the additive homomorphism of BFV, combines them as $c^* = \mathsf{BFV.Add}(c_1, \ldots, c_n)$ so that decryption directly yields the aggregated result, without decrypting individual votes. Decrypts to obtain the election result $r \leftarrow \mathsf{BFV.Dec}(sk, c^*)$, and produces a zero-knowledge proof of correct tallying $\pi \leftarrow \mathsf{ZKP.Prove}(pp, (pk, pbb, r), sk)$ so that anyone can verify the result without learning the secret key. Returns (r, π).
- Verify((pk,pbb,r),π): Checks the tally proof by running $\mathsf{ZKP.Verify}(pp, (pk, pbb, r), \pi)$ and returns its boolean output, allowing any third party to confirm that the published result r correctly reflects the contents of the public bulletin board pbb.
- VerifyVote(b,pbb): Allows a voter to confirm their ballot was correctly recorded by checking whether $H(b) \in \{h \mid (b', h) \in pbb\}$. Using a hash fingerprint rather than the full ballot makes the check lightweight, while collision resistance of H ensures the check is sound.

Note that sk and usk are treated as single secrets above; in practice, both can be threshold-shared among multiple trustees and registrars respectively [89], without affecting the security proofs. Phoenix supports re-voting as if a voter submits a new ballot, only the most recent one is retained on the bulletin board.

```
Setup(1^λ)
  (pk,sk) ← BFV.KeyGen(1^λ)
  pp ← ZKP.Setup(1^λ)
  return (pk,sk,pp)

Register(id)
  (pk_fhs,sk_fhs) ← FHS.Setup(1^λ,1^|τ|,1^ℓ,1^d_max)
  upk ← FHS.KeyGen(pk_fhs,id)
  usk ← sk_fhs
  return (upk,usk)

Verify((pk,pbb,r),π)
  return ZKP.Verify(pp,(pk,pbb,r),π)

VerifyVote(b,pbb)
  b̃ ← strip(b)
  return H(b̃) ∈ {h | (·,h) ∈ pbb}

Vote(id,v,pk,usk)
  upk ← FHS.KeyGen(pk_fhs,id)
  c ← BFV.Enc(pk,v)
  σ ← ZKP.Prove(pp,c,v)
  s ← FHS.Sign(usk,upk,id,(c,σ),1)
  return b ← (id,upk,c,σ,s)

Valid(BB,b,pk)
  (id,upk,c,σ,s) ← b
  e_1 ← ∀x,y.(x,y,·,·,·) ∈ BB ⇒
    (x,y) = (id,upk) ∨ (x ≠ id ∧ y ≠ upk)
  e_2 ← ZKP.Verify(pp,c,σ)
  e_3 ← FHS.Verify(upk,id,(c,σ),s)
  return (e_1 ∧ e_2 ∧ e_3)

Publish(BB)
  pbb ← {(b̃,H(b̃)) | b ∈ Policy(BB), b̃ ← strip(b)}
  return pbb

Tally(BB,sk)
  vbb ← []
  for b ∈ Policy(BB) do
    (id,upk,c,σ,s) ← b
    e_1 ← ZKP.Verify(pp,c,σ)
    e_2 ← FHS.Verify(upk,id,(c,σ),s)
    if (e_1 ∧ e_2) then vbb ← vbb ∪ {c}
  c* ← BFV.Add(vbb)
  r ← BFV.Dec(sk,c*)
  pbb ← Publish(BB)
  π ← ZKP.Prove(pp,(pk,pbb,r),sk)
  return (r,π)
```

Fig. 2: The Phoenix voting system built upon $\mathsf{BFV} = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Add})$, $\mathsf{ZKP} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$, $\mathsf{FHS} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$, and hash function H. The function $\mathsf{strip}(b)$ removes the voter identity from ballot b.

4.1 Concrete Parameters

Phoenix targets $\lambda = 128$ bits of post-quantum security throughout. H is instantiated with SHA-256 (256-bit output, ≥ 128-bit collision resistance, unaffected by Grover's algorithm beyond a factor-two preimage reduction [90]). For BFV, we adopt the Homomorphic Encryption Standard [91] parameter set ($n = 4096$, $\log q = 109$, $t = 2$), achieving ≈ 128-bit security under $\mathsf{RLWE}_{n,q,\chi}$ per the lattice estimator [92]. For ZKP, we follow the ABDLOP instantiation of [73] with $d = 4$, $n = 64$, $\log q \approx 32$, challenge space 2^{256}, and $\beta = 2^{14}$, achieving 128-bit soundness under $\mathsf{MSIS}_{d,m,\beta'}$ and $\mathsf{MLWE}_{d,d,\chi}$. For FHS, we set $n = 512$, $m = 2n\lceil \log q \rceil \approx 22{,}528$, $\log q = 22$, with verification bound B derived from $s \geq \|\widetilde{\mathbf{T}_{\mathbf{A}^*}}\| \cdot \omega(\sqrt{\log m})$ [87,88], context-hiding bound $\tilde{\beta}_{\max} = \beta_{\max} \cdot p_{\text{extra}}(n, m, \log q)$ (Definition 10), and $d_{\max} = 30$ sufficient for Phoenix's tallying circuits.

4.2 Performance and Efficiency Analysis

The core implementation of Phoenix is available in [26]; we benchmarked the Phoenix voting client, which encompasses BFV encryption, ZKP generation, and FHS signing, across four representative platforms: an Android Qualcomm Snapdragon 8 Gen 2 (aarch64) via PRoot Distro[1] on Termux, an Apple M2 (macOS), a Dell Precision 3581 equipped with an Intel Core i7-1355U (Windows 11), the same Intel Core i7-1355U with different RAM running Ubuntu 22.04. As shown in Table 2, ballot creation time is primarily determined by the vote-encoding length $k = \lceil \log_2 n_{cand} \rceil$ augmented by the fixed $\kappa = 16$ credential bits, which shows that the per-ballot cost of Phoenix grows linearly in k, which shows that Phoenix is viable on mobile hardware without requiring native code or multi-threading.

	Dell Precision 3581	Apple Silicon M2	Intel Core i7-1355U	Snapdragon 8 Gen 2
k = 1	2.719×10^{-3}	2.368×10^{-3}	2.631×10^{-3}	2.894×10^{-3}
k = 5	11.817×10^{-3}	9.749×10^{-3}	10.931×10^{-3}	12.120×10^{-3}
k = 10	22.099×10^{-3}	19.385×10^{-3}	20.737×10^{-3}	22.996×10^{-3}
k = 25	58.713×10^{-3}	48.748×10^{-3}	54.841×10^{-3}	68.560×10^{-3}

Table 2: Time in terms of seconds (averaged over three runs) to encrypt, commit, and sign a ballot with a k-bit vote encoding and $\kappa = 16$ credential bits, across four platforms.

5 Security Analysis of Phoenix

Ballot Privacy. We consider the strong correctness, strong consistency, and ideal functionality from [9]. The most fundamental voting privacy property that has been well-studied is BPRIV [9] (no entity can learn the content of ballots), an stronger notion is mb-BPRIV [74] (ballot privacy against a adversary who controls the ballot box), and the strongest one is du-mb-BPRIV (mb-BPRIV where verification is delayed until after tallying) [23], which we focus on it here.

[1] https://github.com/termux/proot-distro

Definition 11 (du-mb-BPRIV [23]). *Let $\mathcal{V}$ be a voting system,* Recover *a recovery algorithm, and $\mathcal{I}=\mathcal{H}\cup\mathcal{D}$ a partition into honest and dishonest voters with $\mathcal{H}_{\mathsf{check}}\subseteq\mathcal{H}$ the verifying subset. A* selection function *for integers $m,n\geq 1$ is a mapping $\pi:[n]\to[m]\cup(\{0,1\}^*\times\{0,1\}^*)$, inducing a board transformation $\bar{\pi}$; the recovery algorithm* $\mathsf{Recover}(BB,BB_1)$ *returns such a π capturing how $\mathcal{A}$ constructed BB from BB_1. We say $\mathcal{V}$ satisfies* du-mb-BPRIV *w.r.t.* Recover *if there exists an efficient simulator* Sim *such that for every efficient adversary $\mathcal{A}$, $\mathsf{Adv}^{\textit{du-mb-bpriv}}_{\mathcal{A},\mathcal{V},\mathsf{Sim}}(\lambda)=\left|\Pr\left[\mathsf{Exp}^0=1\right]-\Pr\left[\mathsf{Exp}^1=1\right]\right|=\mathsf{negl}(\lambda)$, where $\mathsf{Exp}^\beta:=\mathsf{Exp}^{\textit{du-mb-BPRIV},\beta}_{\mathcal{A},\mathcal{V},\mathsf{Sim},\mathsf{Recover}}(\lambda)$ is defined in Figure 3.*

$\mathsf{Exp}^{\textit{du-mb-BPRIV},\beta}_{\mathcal{A},\mathcal{V},\mathsf{Sim},\mathsf{Recover}}(\lambda)$

```
Checked, Happy ← ∅; V, PU, U, CU ← empty
(pd, sd) ← Setup(1^λ)
for id ∈ I do
    (pc, sc) ← Register(id); U[id] ← sc; PU[id] ← pc
    if id ∈ D then CU[id] ← U[id]
BB ← A^{O_voteLR, O_board}(pd, CU, PU, H_check)
if H_check ⊈ V ∨ ValidBoard(BB, pd) = ⊥ then return d ←$ {0,1}
d* ← A()
(r*, Π*) ← A^{O_tally}()
if VerifyTally((pd, pbb, r*), Π*) = ⊥ then return d ←$ {0,1}
d ← A^{O_verify}()
if H_check ⊈ Checked then return d ←$ {0,1}
if H_check ⊈ Happy then return d*
return d
```

$\mathcal{O}_{\mathsf{board}}()$

```
return Publish(BB_β)
```

$\mathcal{O}_{\mathsf{voteLR}}(id,v_0,v_1)$

```
if id ∉ H then return ⊥
for i ∈ {0,1} do (pc, b_i, st^i_pre, st^i_post) ← Vote(pd, pc, v_i)
V[id] ← V[id] || (st^β_pre, st^0_post, v_0)
BB_0 ← BB_0 || (id, (pc, b_0)); BB_1 ← BB_1 || (id, (pc, b_1))
```

$\mathcal{O}_{\mathsf{tally}}()$

```
if β = 0 then (r, Π) ← Tally(BB, pd, sd); return (r, Π)
BB' ← Recover(BB, BB_0, BB_1)
(r, Π) ← Tally(BB', pd, sd)
return (r, Sim(pd, Publish(BB), r))
```

$\mathcal{O}_{\mathsf{verify}}(id)$ for $id\in\mathcal{H}_{\mathsf{check}}$

```
Checked ← Checked ∪ {id}
if VerifyVote(id, st_pre, st_post, BB, pc, sc) = ⊤ then
    Happy ← Happy ∪ {id}
return Happy
```

Fig. 3: The du-mb-BPRIV security experiment. $\mathcal{I}=\mathcal{H}\cup\mathcal{D}$ with $\mathcal{H}_{\mathsf{check}}\subseteq\mathcal{H}$.

In the experiment, $\mathcal{O}_{\mathsf{voteLR}}$ computes ballots for both v_0,v_1, storing them in BB_0,BB_1; the voter state records st^β_{pre} (dependent on the secret bit) and st^0_{post} (always from $\beta=0$, since recovery ensures the tally reflects the left board). After $\mathcal{A}$ outputs board BB, it makes an initial guess d^* *before* seeing the tally. $\mathcal{O}_{\mathsf{tally}}$ (single call): for $\beta=0$ tallies BB directly; for $\beta=1$ recovers $BB'\leftarrow\mathsf{Recover}(BB,BB_0,BB_1)$, tallies BB', and returns the result with a simulated proof. $\mathcal{O}_{\mathsf{verify}}$ runs VerifyVote for voters in $\mathcal{H}_{\mathsf{check}}$. If any expected verification is missing or fails, the experiment returns d^* (or a random bit), ensuring the adversary gains no advantage from tampering that causes detectable complaints.

Theorem 8. *Let* BFV *be IND-CPA secure under* $\mathsf{RLWE}_{n,q,\chi}$, ZKP *be computationally zero-knowledge and knowledge-sound under* MSIS *and* MLWE, FHS *be unforgeable under* $\mathsf{SIS}_{n,m,q,\beta}$ *and statistically context-hiding, and H be collision-resistant. Then for every PPT adversary $\mathcal{A}$ there exists simulator* Sim *and recovery algorithm* Recover *s.t.*

$$\mathsf{Adv}^{\textit{du-mb-bpriv}}_{\mathcal{A},\mathsf{Phoenix},\mathsf{Sim}}(\lambda)\leq\mathsf{Adv}^{\mathsf{ind\text{-}cpa}}_{\mathsf{BFV}}(\lambda)+\mathsf{Adv}^{\mathsf{zk}}_{\mathsf{ZKP}}(\lambda)+\mathsf{Adv}^{\mathsf{ch}}_{\mathsf{FHS}}(\lambda)+\mathsf{Adv}^{\mathsf{cr}}_{H}(\lambda)+\mathsf{negl}(\lambda).$$

In Phoenix, IND-CPA security of BFV makes ciphertexts on BB_0 and BB_1 indistinguishable, so $\mathcal{O}_{\mathsf{board}}$ leaks nothing about β. When $\beta=1$, Recover reconstructs a board BB' consistent with BB_0 via BFV.Add homomorphism and re-randomisation, so tallying BB' yields the same result r as the left board. Zero-knowledge of ZKP lets

Sim produce a tally proof Π^* indistinguishable from a real one without sk. Statistical context-hiding of FHS makes sure signatures on BB_β are identically distributed regardless of β, which prevents board distinction via signature analysis. Finally, collision resistance of H prevents $\mathcal{O}_{\mathsf{verify}}$ and VerifyVote from linking fingerprints to β, so all $\mathcal{H}_{\mathsf{check}}$ voters verify successfully in both worlds.

Machine-checked Privacy Proof. As an interactive theorem prover with support for expressing proofs in the computational model of cryptography, EasyCrypt [25] has been used extensively for checking security mechanisms. The machine-checked privacy proof of Phoenix in EasyCrypt is available at [26], where Theorem 8 corresponds to lemma `Phoenix_du_mb_bpriv` in `PhoenixSecurity.ec`, and the proof model follows [93]. The β=0 and β=1 worlds are encoded as `DU_MB_BPRIV_L` and `DU_MB_BPRIV_R` in `DU_MB_BPRIV.ec`, with board manipulation abstracted via `Recover_Alg`. Each building block (BFV, ZKP, FHS, H) is an abstract `module type` paired with its security game, composed into the protocol in `Phoenix.ec` file:

```
lemma Phoenix_du_mb_bpriv
  (voters : voter_id list) &m :
(* the du-mb-bpriv advantage of Phoenix is upper bounded by: *)
  `| Pr[ DU_MB_BPRIV_L(BFV,ZKP,FHS,H,Sim).init(voters)
       @ &m : true ]
   - Pr[ DU_MB_BPRIV_R(BFV,ZKP,FHS,H,Sim,Rec).init(voters)
       @ &m : true ] |
(* - IND-CPA advantage against BFV *)
  <= Pr[ BFV_INDCPA(BFV,A_BFV).main() @ &m : res ]
(* - ZK advantage against ZKP *)
   + `| Pr[ ZK_Game(ZKP,A_ZK).main(true)  @ &m : res ]
       - Pr[ ZK_Game(ZKP,A_ZK).main(false) @ &m : res ] |
(* - context-hiding advantage against FHS *)
   + `| Pr[ FHS_CH_Game(FHS,A_FHS).main(true)  @ &m : res ]
       - Pr[ FHS_CH_Game(FHS,A_FHS).main(false) @ &m : res ] |
(* - collision-resistance advantage against H *)
   + Pr[ CR_Game(H,A_CR).main() @ &m : res ].
proof. admit. qed.
```

Strong Consistency. This property demands that for any (adversarially produced) bulletin board for which each individual ballot is valid, the result provided by the Tally algorithm is the correct one. The original definition associated voter identities with ballots; here, we use their public credentials instead.

Definition 12. *A voting scheme is* strongly consistent *if there exists:*

- *An extraction algorithm* Extract(b,sk) *that provides the upk with the value computed by decrypting of the ciphertext contained in the ballot b with the secret sk; and*
- *A ballot validation algorithm* ValidInd(b,pk) *returns true iff ballot b is "well-formed" with respect to some notion of well-formedness determined in advance by election.*

These algorithms must satisfy the following conditions:

- *For any* (pk,sk) *obtained from* Setup(λ)*, and any* (id,v,usk) *with* upk *trivially computed from* usk*, if* $b \leftarrow$ Vote(id,v,pk,usk) *then* Extract(b,sk) *returns* (upk,v) *with overwhelming probability.*
- *For any adversarially produced* (BB, b)*, if* Valid(BB, b, pk) *returns true, then* ValidInd(b,pk) *returns true as well.*

– *For any adversary $\mathcal{B}$ that returns a ballot box with ballots that satisfy* ValidInd, *the experiment* $\mathsf{Exp}^{\mathsf{consis}}_{\mathcal{B},\mathcal{V},I}(\lambda)$ *specified in Figure 4 returns true with a probability negligible in the security parameter.*

Theorem 9. *Let* BFV *be correct and IND-CPA-secure under* $\mathsf{RLWE}_{n,q,\chi}$, *let* ZKP *be complete and knowledge-sound under* MSIS, *and let* FHS *be unforgeable under* $\mathsf{SIS}_{n,m,q,\beta}$. *Define* $\mathsf{Extract}(b,sk)$ *to parse* $b=(id,upk,c,\sigma,s)$ *and return the* $(upk,\mathsf{BFV.Dec}(sk,c))$, *and* $\mathsf{ValidInd}(b,pk)$ *to return* 1 *iff* $\mathsf{ZKP.Verify}(pp,c,\sigma)=1$ *and* $\mathsf{FHS.Verify}(upk,id,(c,\sigma),s)=1$. *Then for every PPT adversary* $\mathcal{B}$, *we have* $\Pr\left[\mathsf{Exp}^{\mathsf{consis}}_{\mathcal{B},\mathsf{Phoenix},\mathcal{I}}(\lambda)=1\right] \leq \mathsf{Adv}^{\mathsf{ks}}_{\mathsf{ZKP}}(\lambda)+\mathsf{negl}(\lambda)$, *where the negligible term accounts for the probability that* BFV *decryption fails on a correctly encrypted plaintext.*

Strong consistency of Phoenix follows from the tally path being deterministic once BB is fixed. If every $b \in BB$ satisfies ValidInd, knowledge soundness of ZKP guarantees each c encrypts a valid v, and each s binds (c,σ) to upk. Correctness of BFV makes sure Extract recovers (upk,v) w.o.p. Tally then applies Policy, aggregates via BFV.Add, and decrypts, which yields $\sum v_i = \mathsf{Count} \circ \mathsf{Policy}$ on the extracted votes by additive homomorphism. Hence $r=r'$ in $\mathsf{Exp}^{\mathsf{consis}}$ holds with overwhelming probability.

Strong Correctness. This is the guarantee that an honestly created ballot will not be rejected by a ballot validation algorithm.

Definition 13 (Strong Correctness). *A voting scheme* $\mathcal{V}$ *is* strongly correct *if the advantage of any efficient adversary* $\mathcal{B}$, *defined by* $\mathcal{A}^{\mathsf{corr}}_{\mathcal{B},\mathcal{V},I}(\lambda) = \Pr[\mathsf{Exp}^{\mathsf{corr}}_{\mathcal{B},\mathcal{V},I}(\lambda)=1]$ *(where* $\mathsf{Exp}^{\mathsf{corr}}_{\mathcal{B},\mathcal{V},I}(\lambda)$ *is defined in Figure 4) is negligible as a function of* λ.

$\mathsf{Exp}^{\mathsf{consis}}_{\mathcal{B},\mathcal{V},I}(\lambda)$

```
uL ← empty
(pk,sk) ← Setup(1^λ)
∀id ∈ I do uL.[id] ← Register(id)
BB ← B(pk,uL)
e ← ∀b ∈ BB. ValidInd(b,pk)
(r,Π) ← Tally(BB,sk)
for i in 1..|BB| do
    dbb[i] ← Extract(BB[i],sk)
r' ← Count∘Policy(dbb)
return (r ≠ r' ∧ e)
```

$\mathsf{Exp}^{\mathsf{corr}}_{\mathcal{B},\mathcal{V},I}(\lambda)$

```
uL ← empty
(pk,sk) ← Setup(1^λ)
∀id ∈ I do uL.[id] ← Register(id)
(id,v,BB) ← B(pk,uL)
e ← false
if (id ∈ I) then
    (usk,upk) ← uL.[id]
    e1 ← ∀x.∀y. (x,y,⋆,⋆) ∈ BB ⟹
        (x,y) = (id,upk) ∨ (x ≠ id ∧ y ≠ upk)
    b ← Vote(id,v,pk,usk)
    e2 ← Valid(BB,b,pk)
    e ← e1 ∧ ¬e2
return e
```

Fig. 4: Security experiments for strong consistency (left, $\mathsf{Exp}^{\mathsf{consis}}$) and strong correctness (right, $\mathsf{Exp}^{\mathsf{corr}}$). The adversary $\mathcal{B}$ wins $\mathsf{Exp}^{\mathsf{consis}}$ if tallying produces a result inconsistent with the extracted ballots despite all ballots being individually valid. The adversary wins $\mathsf{Exp}^{\mathsf{corr}}$ if it can submit a ballot bulletin board that renders an honest voter's fresh ballot invalid.

Theorem 10. *Let* BFV *be correct, let* ZKP *be complete, and let* FHS *be correct (i.e.* FHS.Verify *accepts honestly generated signatures with overwhelming probability). Then for every PPT adversary* $\mathcal{B}$, $\mathsf{Adv}^{\mathsf{corr}}_{\mathcal{B},\mathsf{Phoenix},\mathcal{I}}(\lambda) = \Pr\left[\mathsf{Exp}^{\mathsf{corr}}_{\mathcal{B},\mathsf{Phoenix},\mathcal{I}}(\lambda)=1\right] \leq \mathsf{negl}(\lambda)$.

In Phoenix, an honest ballot $b \leftarrow \mathsf{Vote}(id, v, pk, usk)$ is constructed by running BFV.Enc, ZKP.Prove, and FHS.Sign with correctly generated keys, so each component is well-formed by the correctness of the respective primitive. The credential-consistency check e_1 in Valid holds because the adversary $\mathcal{B}$ may choose BB but the experiment enforces that no conflicting (id,upk) pair appears unless $\mathcal{B}$ itself placed it, and the check is precisely that honest (id,upk) is not contradicted. Completeness of ZKP makes sure $e_2 = 1$, and correctness of FHS makes sure $e_3 = 1$. Hence $\mathsf{Valid}(BB,b,pk) = 1$ with overwhelming probability, and $\mathcal{B}$ cannot cause $e_1 \wedge \neg e_2$ to hold.

Ideal Functionality. This property states the voting protocol execution is indistinguishable from interaction with a trusted party.

Definition 14 (Ideal Voting Functionality). *Let $\mathcal{I}$ be a set of voter identities, $\mathbb{V}$ a set of possible votes, and $\rho : (\mathcal{I} \times \mathbb{V})^* \to \mathcal{R}$ a result function mapping sequences of identity-vote pairs to a result space $\mathcal{R}$. The ideal functionality $\mathcal{F}_{\text{voting}}(\rho)$, interacting with an environment $\mathfrak{E}$ (controlling honest voters) and an ideal-world adversary (simulator) $\mathcal{S}$ (acting on behalf of corrupted voters), maintains an initially empty list L of identity-vote pairs and proceeds as follows:*

1. ***Vote:*** *On input* $\mathsf{vote}(id,v)$ *from $\mathfrak{E}$ or $\mathcal{S}$, append (id,v) to L and send* $\mathsf{ack}(id)$ *to $\mathcal{S}$.*
2. ***Tally:*** *On input* tally *from $\mathcal{S}$, compute $r \leftarrow \rho(L)$ and return r to both $\mathfrak{E}$ and $\mathcal{S}$, then halt.*

The full ideal-world execution, in which $\mathfrak{E}$, $\mathcal{S}$, and $\mathcal{F}_{\text{voting}}(\rho)$ interact and $\mathfrak{E}$ outputs a single bit, is denoted $\beta \leftarrow \mathsf{idealexec}(\mathfrak{E} \| \mathcal{S} \| \mathcal{F}_{\text{voting}}(\rho))$.

Theorem 11. *If* Phoenix *is* du-mb-BPRIV *(Theorem 8), strongly consistent (Theorem 9), and strongly correct (Theorem 10), then for every PPT environment $\mathfrak{E}$ there exists a PPT simulator $\mathcal{S}$ such that*

$$\left|\Pr[\mathsf{realexec}(\mathfrak{E} \| \mathcal{A} \| \mathsf{Phoenix}) = 1] - \Pr[\mathsf{idealexec}(\mathfrak{E} \| \mathcal{S} \| \mathcal{F}_{\text{voting}}(\rho)) = 1]\right| \leq \mathsf{negl}(\lambda).$$

By the composition theorem of Cortier et al. [9], any scheme satisfying mb-BPRIV, strong correctness, and strong consistency realises $\mathcal{F}_{\text{voting}}(\rho)$: strong correctness lets $\mathcal{S}$ simulate the honest voting interface; strong consistency makes sure the real-world tally matches ρ on extracted votes; and mb-BPRIV lets $\mathcal{S}$ simulate the adversary's view (board and tally proof) from r alone. Since Phoenix satisfies all three by Theorems 8–10, the composition applies directly.

Verifiability. Generally, it comes from the fact that voters can verify their ballots were recorded and anyone can verify correct tallying, but we consider the *Strong Verifiability* notion by [75] here, which covers broader spectrum.

Definition 15 (Strong Verifiability). *Let $\mathcal{V} =$* (Setup, Credential, Vote, Validate, Box, VerifyVote, Tally, Verify) *be a voting protocol with admissible vote space $\mathbb{V}$, result space $\mathcal{R}$, and result function $\rho : \mathbb{V}^\tau \to \mathcal{R}$ admitting partial tallying via a commutative operation $\oplus_R$, i.e. $\rho(S_1 \cup S_2) = \rho(S_1) \oplus_R \rho(S_2)$ for disjoint vote lists $S_1, S_2 \subseteq \mathbb{V}^*$. Both experiments maintain: a public credential set L; a table U of registered tuples (id,upk,usk); a set CU of corrupted voter pairs (id,upk); a list $HVote$ of triples (id,v,b) from honest voting queries (last ballot per id only); and $Checked \subseteq HVote$ where* $\mathsf{CheckedVoter}(id,v,b) = 1$. *The oracles are: $\mathcal{O}_{\mathsf{reg}}(id)$: honestly generates $(upk, usk) \leftarrow \mathsf{Credential}(1^\lambda, id)$, returns*

upk, updates L,U. $\mathcal{O}_{\mathsf{corrupt}}(id)$*: reveals* (upk,usk) *for registered id, updates* CU. $\mathcal{O}_{\mathsf{vote}}(id,v)$*: for honest, uncorrupted id with* $v \in \mathbb{V}$*, returns* $b \leftarrow \mathsf{Vote}(id,upk,usk,v)$*, updates HVote.* $\mathcal{O}_{\mathsf{cast}}(id,b)$*: runs* $\mathsf{Box}(BB,b)$ *on an honest bulletin board.*

The experiments are given in Figure 5. In both, the adversary wins if it produces a verified tally that cannot be expressed as $\rho(\{v_i^E\}) \oplus_R \rho(\{v_i^A\}) \oplus_R \rho(\{v_i^B\})$*, where* $\{v_i^E\}$ *are votes from Checked,* $\{v_i^A\}$ *are from some subset of* $HVote \setminus Checked$*, and* $\{v_i^B\} \subseteq \mathbb{V}$ *with* $|\{v_i^B\}| \leq |CU|$*. We say* $\mathcal{V}$ *achieves* strong verifiability *if for all PPT* $\mathcal{A}$*, both* $\Pr[\mathsf{Exp}^{\mathsf{ver\text{-}b}}_{\mathcal{A},\mathcal{V}}(\lambda)=1]$ *and* $\Pr[\mathsf{Exp}^{\mathsf{ver\text{-}r}}_{\mathcal{A},\mathcal{V}}(\lambda)=1]$ *are negligible, i.e. under the assumption that the registration authority and bulletin board are not simultaneously dishonest.*

$\mathsf{Exp}^{\mathsf{ver\text{-}b}}_{\mathcal{A},\mathcal{V}}(\lambda)$

$L,U,CU,HVote \leftarrow \emptyset$
$(pk,sk) \leftarrow \mathsf{Setup}(1^\lambda)$
$(BB,\rho,\Pi) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{reg}},\mathcal{O}_{\mathsf{corrupt}},\mathcal{O}_{\mathsf{vote}}}(pk)$
if $\mathsf{Verify}(BB,\rho,\Pi)=0 \vee \rho=\perp$ **then return** 0
if $\exists\{(id_i^A,v_i^A,\star)\} \subseteq HVote \setminus Checked, \exists\{v_i^B\} \subseteq \mathbb{V}, |\{v_i^B\}| \leq |CU|$:
  $\rho=\rho(\{v_i^E\}) \oplus_R \rho(\{v_i^A\}) \oplus_R \rho(\{v_i^B\})$ **then return** 0
return 1

$\mathsf{Exp}^{\mathsf{ver\text{-}r}}_{\mathcal{A},\mathcal{V}}(\lambda)$

$L,U,CU,HVote \leftarrow \emptyset$
$(pk,sk) \leftarrow \mathsf{Setup}(1^\lambda)$
$(\rho,\Pi) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{vote}},\mathcal{O}_{\mathsf{corrupt}},\mathcal{O}_{\mathsf{cast}}}(pk)$
if $\mathsf{Verify}(BB,\rho,\Pi)=0 \vee \rho=\perp$ **then return** 0
if $\exists\{(id_i^A,v_i^A,\star)\} \subseteq HVote \setminus Checked, \exists\{v_i^B\} \subseteq \mathbb{V}, |\{v_i^B\}| \leq |CU|$:
  $\rho=\rho(\{v_i^E\}) \oplus_R \rho(\{v_i^A\}) \oplus_R \rho(\{v_i^B\})$ **then return** 0
return 1

Fig. 5: $\mathsf{Exp}^{\mathsf{ver\text{-}b}}$ (left): $\mathcal{A}$ controls trustee, voters, and bulletin board; registrar is honest. $\mathsf{Exp}^{\mathsf{ver\text{-}r}}$ (right): $\mathcal{A}$ controls trustee, voters, and registrar; bulletin board is honest ($\mathcal{A}$ adds ballots only via $\mathcal{O}_{\mathsf{cast}}$, does not output BB).

Theorem 12. *Let* H *be collision-resistant, let* ZKP *be knowledge-sound under* MSIS*, let* FHS *be existentially unforgeable under* $\mathsf{SIS}_{n,m,q,\beta}$*, and let* BFV *be correct. Then for every PPT adversary* $\mathcal{A}$*,* $\Pr\left[\mathsf{Exp}^{\mathsf{ver\text{-}b}}_{\mathcal{A},\mathsf{Phoenix}}(\lambda)=1\right] \leq \mathsf{Adv}^{\mathsf{euf}}_{\mathsf{FHS}}(\lambda)+\mathsf{Adv}^{\mathsf{ks}}_{\mathsf{ZKP}}(\lambda)+\mathsf{Adv}^{\mathsf{cr}}_{H}(\lambda)+\mathsf{negl}(\lambda)$*, and likewise* $\Pr[\mathsf{Exp}^{\mathsf{ver\text{-}r}}_{\mathcal{A},\mathsf{Phoenix}}(\lambda)=1] \leq \mathsf{negl}(\lambda)$ *under the same assumptions.*

Individual verifiability follows from collision resistance of H: a voter checks that $H(b)$ appears on pbb, and any substitution or omission requires a collision or a detectable removal. Universal verifiability follows from knowledge soundness of π: $\mathsf{Verify}((pk,pbb,r),\pi)=1$ guarantees r is the correct decryption of the homomorphic aggregate on pbb, so no dishonest trustee can publish an incorrect result. Eligibility verifiability follows from existential unforgeability of FHS: every accepted ballot must carry a valid signature under a registered upk, bounding adversarial ballot contributions to at most $|CU|$. Together, these make sure no decomposition $r=\rho(\{v_i^E\}) \oplus_R \rho(\{v_i^A\}) \oplus_R \rho(\{v_i^B\})$ with $|\{v_i^B\}| \leq |CU|$ can be violated, which yields strong verifiability.

6 Future Directions

Unlike other post-quantum schemes [4, 10–19], we plan to deploy an actual production-ready implementation of Phoenix as an online web-application voting platform. Also, we aim to eliminate the distributed key generation phase on real-world deployment of Phoenix via silent-setup threshold cryptosystems [94]; however, current constructions rely on witness encryption, which remains inefficient and quantum-vulnerable [95–97]. Designing a post-quantum silent-setup threshold public key scheme that can support homomorphic operations is an open problem. In addition, except using signatures on randomizable ciphertexts [98], no solution has been proposed to achieve strong receipt-freeness [99], so using the same technique on PQ voting can be progressive, but designing post-quantum signatures on randomizable ciphertexts remains as an open problem.

A Appendix

A.1 Proof of Theorem 1

Proof. Let $(pk, sk) \leftarrow \mathsf{Setup}(1^\lambda, 1^{|\tau|}, 1^\ell, 1^{d_{\max}})$ and $vk \leftarrow \mathsf{KeyGen}(pk, \tau)$. For each $k \in \{0,1\}$, the $\mathsf{SampleLeft}$ algorithm produces a short matrix $[\mathbf{R}_{k,1}^\top \,|\, \mathbf{R}_{k,2}^\top]^\top$ satisfying

$$\mathbf{A}_\tau \begin{bmatrix} \mathbf{R}_{k,1} \\ \mathbf{R}_{k,2} \end{bmatrix} = \mathbf{D}_k + [k\mathbf{I}_n \,|\, \cdots \,|\, k\mathbf{I}_n]\mathbf{G}_n. \tag{1}$$

For a message $\mu_i \in \{0,1\}$ at index $i \in [\ell]$, setting $k = \mu_i$, the Unpack procedure computes

$$\begin{bmatrix} \mathbf{R}_{\mu_i,1,i} \\ \mathbf{R}_{\mu_i,2,i} \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{k,1} \\ \mathbf{R}_{k,2} \end{bmatrix} \cdot \mathbf{G}_n^{-1}(\mathbf{P}_i \mathbf{G}_n).$$

Right-multiplying both sides of (1) by $\mathbf{G}_n^{-1}(\mathbf{P}_i\mathbf{G}_n)$ yields

$$\begin{aligned} \mathbf{A}_\tau \begin{bmatrix} \mathbf{R}_{\mu_i,1,i} \\ \mathbf{R}_{\mu_i,2,i} \end{bmatrix} &= \big(\mathbf{D}_k + [k\mathbf{I}_n \,|\, \cdots \,|\, k\mathbf{I}_n]\mathbf{G}_n\big) \cdot \mathbf{G}_n^{-1}(\mathbf{P}_i\mathbf{G}_n) \\ &= \mathbf{D}_{\mu_i}\mathbf{G}_n^{-1}(\mathbf{P}_i\mathbf{G}_n) + [k\mathbf{I}_n \,|\, \cdots \,|\, k\mathbf{I}_n]\mathbf{G}_n\mathbf{G}_n^{-1}(\mathbf{P}_i\mathbf{G}_n) \\ &= \mathbf{D}_{\mu_i}\mathbf{G}_n^{-1}(\mathbf{P}_i\mathbf{G}_n) + [k\mathbf{I}_n \,|\, \cdots \,|\, k\mathbf{I}_n]\mathbf{P}_i\mathbf{G}_n \\ &= \mathbf{D}_{\mu_i}\mathbf{G}_n^{-1}(\mathbf{P}_i\mathbf{G}_n) + \mu_i\mathbf{G}_n, \end{aligned} \tag{2}$$

where the last equality uses the property of the extended unit matrix $\mathbf{P}_i = [\mathbf{0}_n \,|\, \cdots \,|\, \mathbf{I}_n \,|\, \cdots \,|\, \mathbf{0}_n]$, which extracts the i-th block so that $[k\mathbf{I}_n \,|\, \cdots \,|\, k\mathbf{I}_n]\mathbf{P}_i = k\mathbf{I}_n$ and hence $[k\mathbf{I}_n \,|\, \cdots \,|\, k\mathbf{I}_n]\mathbf{P}_i\mathbf{G}_n = k\mathbf{G}_n = \mu_i\mathbf{G}_n$.

The public matrix associated with the identity circuit id_i is $\mathbf{D}_{\mathsf{id}_i} = \mathbf{D}_{\mu_i}\mathbf{G}_n^{-1}(\mathbf{P}_i\mathbf{G}_n)$. Since $\mathsf{SampleLeft}$ makes sure $\|[\mathbf{R}_{k,1}^\top \,|\, \mathbf{R}_{k,2}^\top]^\top\|_\infty \le s\sqrt{m}$ and $\mathbf{G}_n^{-1}(\mathbf{P}_i\mathbf{G}_n) \in \{0,1\}^{m\times m}$, we have $\|[\mathbf{R}_{\mu_i,1,i}^\top \,|\, \mathbf{R}_{\mu_i,2,i}^\top]^\top\|_\infty \le m \cdot s\sqrt{m} \le B$ for the chosen parameters. Together with (2), the verification check $\mathbf{A}_\tau\mathbf{R}_{\sigma_i} = \mathbf{D}_{\mathsf{id}_i} + \mu_i\mathbf{G}_n$ and $\|\mathbf{R}_{\sigma_i}\|_\infty \le B$ both hold, so $\mathsf{Verify}(vk, \tau, \sigma_i, \mathsf{id}_i) = 1$. □

A.2 Proof of Theorem 2

Proof. We proceed by induction on the circuit structure. Since every Boolean circuit can be expressed using NAND gates, it suffices to verify correctness for a single NAND gate $g = (u, v, w)$ with inputs $x, y \in \{0,1\}$.

Suppose that at the input wires of gate g, we have signatures $\mathbf{R}_u, \mathbf{R}_v \in \mathbb{Z}^{2m\times m}$ satisfying

$$\mathbf{A}_\tau\mathbf{R}_u = \mathbf{D}_u\mathbf{G}_n^{-1}(\mathbf{P}_{x_i}\mathbf{G}_n) + x\mathbf{G}_n \quad \text{and} \quad \mathbf{A}_\tau\mathbf{R}_v = \mathbf{D}_v\mathbf{G}_n^{-1}(\mathbf{P}_{y_j}\mathbf{G}_n) + y\mathbf{G}_n.$$

The base case holds by signing correctness (Theorem 1).

The Eval algorithm computes $\hat{\mathbf{D}}_u = \mathbf{G}_n^{-1}(\mathbf{D}_u\mathbf{G}_n^{-1}(\mathbf{P}_{x_i}\mathbf{G}_n)) \in \{0,1\}^{m\times m}$, the output signature $\mathbf{R}_w = \mathbf{R}_v\hat{\mathbf{D}}_u - y\mathbf{R}_u$, and the public matrix $\mathbf{D}_w = \mathbf{D}_v\mathbf{G}_n^{-1}(\mathbf{P}_{y_j}\mathbf{G}_n)\hat{\mathbf{D}}_u - \mathbf{G}_n$.

We verify:

$$\begin{aligned}
\mathbf{A}_\tau \mathbf{R}_w &= \mathbf{A}_\tau(\mathbf{R}_v \hat{\mathbf{D}}_u - y\mathbf{R}_u) \\
&= (\mathbf{A}_\tau \mathbf{R}_v)\hat{\mathbf{D}}_u - y(\mathbf{A}_\tau \mathbf{R}_u) \\
&= \big(\mathbf{D}_v \mathbf{G}_n^{-1}(\mathbf{P}_{y_j}\mathbf{G}_n) + y\mathbf{G}_n\big)\hat{\mathbf{D}}_u - y\big(\mathbf{D}_u \mathbf{G}_n^{-1}(\mathbf{P}_{x_i}\mathbf{G}_n) + x\mathbf{G}_n\big) \\
&= \mathbf{D}_v \mathbf{G}_n^{-1}(\mathbf{P}_{y_j}\mathbf{G}_n)\hat{\mathbf{D}}_u + y\mathbf{G}_n\hat{\mathbf{D}}_u - y\mathbf{D}_u \mathbf{G}_n^{-1}(\mathbf{P}_{x_i}\mathbf{G}_n) - xy\mathbf{G}_n \\
&= \mathbf{D}_v \mathbf{G}_n^{-1}(\mathbf{P}_{y_j}\mathbf{G}_n)\hat{\mathbf{D}}_u + y\mathbf{D}_u \mathbf{G}_n^{-1}(\mathbf{P}_{x_i}\mathbf{G}_n) - y\mathbf{D}_u \mathbf{G}_n^{-1}(\mathbf{P}_{x_i}\mathbf{G}_n) - xy\mathbf{G}_n \\
&= \mathbf{D}_v \mathbf{G}_n^{-1}(\mathbf{P}_{y_j}\mathbf{G}_n)\hat{\mathbf{D}}_u - xy\mathbf{G}_n \\
&= \mathbf{D}_w + \mathbf{G}_n - xy\mathbf{G}_n \\
&= \mathbf{D}_w + (1-xy)\mathbf{G}_n = \mathbf{D}_w + (x\,\mathsf{NAND}\,y)\mathbf{G}_n,
\end{aligned}$$

where in the fifth equality we used $\mathbf{G}_n\hat{\mathbf{D}}_u = \mathbf{G}_n\mathbf{G}_n^{-1}(\mathbf{D}_u\mathbf{G}_n^{-1}(\mathbf{P}_{x_i}\mathbf{G}_n)) = \mathbf{D}_u\mathbf{G}_n^{-1}(\mathbf{P}_{x_i}\mathbf{G}_n)$.

For the noise bound, since $\hat{\mathbf{D}}_u \in \{0,1\}^{m\times m}$ and $|y| \le 1$, we have $\|\mathbf{R}_w\|_\infty \le m\|\mathbf{R}_v\|_\infty + \|\mathbf{R}_u\|_\infty$. By induction over a circuit of depth $d_{\max}$, the noise grows as $\|\mathbf{R}_f\|_\infty \le (m+1)^{d_{\max}} \cdot \beta_{\text{init}} \le B$ for appropriately chosen parameters.

Composed evaluation follows by the same inductive argument: if σ_j certifies $h_j(\boldsymbol{\mu})$ under public matrix $\mathbf{D}_{h_j}$, then evaluating g on $(\sigma_1, \dots, \sigma_s)$ yields σ^* certifying $g(h_1(\boldsymbol{\mu}),\dots,h_s(\boldsymbol{\mu}))$ under $\mathbf{D}_{g\circ\bar{h}}$, provided the total depth does not exceed $d_{\max}$. □

A.3 Proof of Theorem 3

Proof. The evaluated signature $\sigma_f = (\mathbf{R}_f, f(\boldsymbol{\mu}), f)$ has its dominant component $\mathbf{R}_f \in \mathbb{Z}_q^{2m\times m}$. Each entry of $\mathbf{R}_f$ can be represented with $\lceil \log q \rceil$ bits, so $|\mathbf{R}_f| = 2m^2\lceil \log q \rceil$. Recall $m = O(n \cdot d_{\max})$ and $q = n^{O(d_{\max})}$ for polylogarithmic-depth circuits, so $\lceil \log q \rceil = O(d_{\max}\log n)$ and $|\mathbf{R}_f| = \text{poly}(\lambda, d_{\max})$.

The representation size of $\mathbf{R}_f$ is *independent* of the circuit size $|f|$ and of the number of inputs ℓ. The dataset size ℓ enters only through the vector encoding's use of extended unit matrices $\mathbf{P}_i \in \mathbb{Z}^{\ell n\times n}$; however, the dimension of the gadget matrix $\mathbf{G}_n$ and thus the dimension of $\mathbf{R}_f$ depend on ℓ only logarithmically via the recursive packing technique, which yields $\|\sigma_f\| \le p(\lambda)\cdot\log\ell$ for a fixed polynomial p. □

A.4 Proof of Theorem 4

Proof. We reduce the unforgeability of FHS to the $\mathsf{SIS}_{q,n,m,\beta}$ problem. Given a random matrix $\mathbf{A}^* \in \mathbb{Z}_q^{n\times m}$ from a SIS challenger, we construct an algorithm $\mathcal{B}$ that simulates the security experiment for an adversary $\mathcal{A}$ and extracts a SIS solution from any successful forgery. The algorithm $\mathcal{B}$ first guesses which type of forgery (Type I or Type II) the adversary will produce.

Type I forgery (fresh tag τ^*). The algorithm $\mathcal{B}$ sets up the public key using the partitioning technique. It samples $\mathbf{U},\mathbf{U}_0,\mathbf{U}_1 \leftarrow\$ \{-1,1\}^{m\times m}$, $\mathbf{R},\mathbf{R}' \leftarrow\$ \{-1,1\}^{m\times m}$, and the partition hash parameters $(\mathbf{z},\boldsymbol{\alpha},\boldsymbol{\beta})$. It sets $\mathbf{A} = \mathbf{A}^*\mathbf{U}$, $\mathbf{D}_k = \mathbf{A}^*\mathbf{U}_k$ for $k \in \{0,1\}$, $\mathbf{B} = \mathbf{A}^*\mathbf{R} + \mathbf{E}$, and $\mathbf{B}' = \mathbf{A}^*\mathbf{R}' + \mathbf{E}'$, where $\mathbf{E},\mathbf{E}'$ encode the partition hash parameters. By Lemma 2.6 (leftover hash lemma), $(\mathbf{A},\mathbf{D}_0,\mathbf{D}_1,\mathbf{B},\mathbf{B}')$ are statistically close to uniform. The element $\mathbf{z} = \mathbf{A}^*\cdot\mathbf{r}_0$ for $\mathbf{r}_0 \leftarrow\$ \{0,1\}^m$ is computed honestly.

For each signing query (τ,μ_i,i) with $h_{\mathbf{z},\boldsymbol{\alpha},\boldsymbol{\beta}}(\tau) \neq \mathbf{0}$, the algorithm $\mathcal{B}$ runs $\mathbf{R}_\tau \leftarrow$ TrapEval to obtain a $\mathbf{G}_{\nu n}$-trapdoor for $\mathbf{A}_\tau = [\mathbf{A}^* \mid \mathbf{A} + \mathbf{B}_\tau]$. Using SampleDO, it

computes pre-signatures satisfying (1) and then applies Unpack to derive σ_i. The statistical closeness of $\mathsf{SampleDO}$ and $\mathsf{SampleLeft}$ (for the common Gaussian parameter s) makes sure that the signatures are indistinguishable from honestly generated ones. If $h_{\mathbf{z},\boldsymbol{\alpha},\boldsymbol{\beta}}(\tau)=\mathbf{0}$, the simulator aborts.

When $\mathcal{A}$ outputs a Type I forgery $(\tau^*,\sigma^*_{f^*},\mu^*,f^*)$ with $\tau^*\neq\tau_j$ for all queried τ_j, the event $h_{\mathbf{z},\boldsymbol{\alpha},\boldsymbol{\beta}}(\tau^*)=\mathbf{0}$ occurs with probability at least $0.5/q^u$, which is non-negligible. In this case, $\mathbf{B}_{\tau^*}=\mathbf{A}^*\mathbf{R}_{\tau^*}$ (the $\mathbf{E}_{\tau^*}\mathbf{G}_{\nu n}$ term vanishes) and the verification equation gives

$$\mathbf{A}^*\big(\mathbf{R}^*_{f^*,1}+(\mathbf{U}+\mathbf{R}_{\tau^*})\mathbf{R}^*_{f^*,2}-\mathbf{U}^*_{f^*}\big)=(k+\mu^*)\mathbf{G}_n,$$

where $\mathbf{D}^*_{f^*}=\mathbf{A}^*\mathbf{U}^*_{f^*}+k\mathbf{G}_n$ by the homomorphic structure (Lemma 4.2 of [22]). Since $\mathbf{R}_{\tau^*}$ is information-theoretically hidden from $\mathcal{A}$, the vector $\mathbf{w}=\mathbf{R}^*_{f^*,1}+(\mathbf{U}+\mathbf{R}_{\tau^*})\mathbf{R}^*_{f^*,2}-\mathbf{U}^*_{f^*}$ is non-zero with overwhelming probability. If $k+\mu^*=0$, then $\mathbf{A}^*\mathbf{w}=\mathbf{0}$ directly yields a SIS solution; otherwise, $\mathbf{w}'=\mathbf{w}\cdot\mathbf{T}_{\mathbf{G}_n}$ satisfies $\mathbf{A}^*\mathbf{w}'=\mathbf{0}$, with $\|\mathbf{w}'\|\leq\beta$.

Type II forgery (previously queried tag $\tau^*=\tau_j$, but $\mu^*\neq f^*(\boldsymbol{\mu}_j)$). The algorithm $\mathcal{B}$ randomly guesses the target tag τ^* and sets $\mathbf{A}=\mathbf{A}^*\mathbf{U}-\mathbf{B}_{\tau^*}$ so that $\mathbf{A}_{\tau^*}=[\mathbf{A}^*\,|\,\mathbf{A}^*\mathbf{U}]$. It samples pre-signatures $[\mathbf{R}^\top_{b,1}\,|\,\mathbf{R}^\top_{b,2}]^\top\leftarrow(\mathcal{D}_{\mathbb{Z}^m,s})^{2m}$ for $b\in\{0,1\}$ and defines $\mathbf{D}_b=[\mathbf{A}^*\,|\,\mathbf{A}^*\mathbf{U}]\cdot[\mathbf{R}^\top_{b,1}\,|\,\mathbf{R}^\top_{b,2}]^\top-[b\mathbf{I}_n\,|\cdots|\,b\mathbf{I}_n]\mathbf{G}_n$. We already know $(\mathbf{A},\mathbf{D}_0,\mathbf{D}_1)$ are statistically close to uniform. Signing queries for τ^* are answered using Unpack on the pre-generated pre-signatures. Other tags cause an abort.

For a Type II forgery, $\mathcal{B}$ also computes the honest evaluated signature $\mathbf{R}_{f^*}$ from the pre-signatures. Subtracting the forged verification equation from the honest one yields

$$\mathbf{A}^*\big(\mathbf{R}_{f^*,1}-\mathbf{R}^*_{f^*,1}+\mathbf{U}(\mathbf{R}_{f^*,2}-\mathbf{R}^*_{f^*,2})\big)=\big(f^*(\boldsymbol{\mu}_j)-\mu^*\big)\mathbf{G}_n.$$

Since $f^*(\boldsymbol{\mu}_j)\neq\mu^*$ and the pre-signatures are sampled from a continuous Gaussian distribution hidden from $\mathcal{A}$, the left-hand vector is non-zero with overwhelming probability. The resulting non-zero short vector (or its product with $\mathbf{T}_{\mathbf{G}_n}$) yields a valid SIS solution.

The guess of forgery type succeeds with probability $1/2$, and the partition succeeds with non-negligible probability. The total advantage of $\mathcal{B}$ in solving SIS is therefore a non-negligible fraction of $\mathcal{A}$'s advantage, which completes our reduction. □

A.5 Proof of Theorem 5

Proof. Since $\mathsf{Verify}(vk,\tau,\sigma_f,f)=1$, we have $\mathbf{A}_\tau\mathbf{R}_f=\mathbf{D}_f+y\mathbf{G}_n$ and $\|\mathbf{R}_f\|_\infty\leq B\leq\beta_{\max}$. The Hide procedure forms

$$\mathbf{H}_f=[\mathbf{A}_\tau\,|\,\mathbf{D}_f+(y-1)\mathbf{G}_n]\in\mathbb{Z}_q^{n\times 3m}.$$

Observe that $\mathbf{D}_f+(y-1)\mathbf{G}_n=\mathbf{A}_\tau\mathbf{R}_f+(-1)\mathbf{G}_n$, so the second block of $\mathbf{H}_f$ has the form $\mathbf{A}_\tau\mathbf{R}_f+h\mathbf{G}_n$ with $h=-1\neq 0$. This means $\mathbf{R}_f$ is a valid $\mathbf{G}_n$-trapdoor for $\mathbf{H}_f$ with non-zero tag $h=-1$ (in the sense of Definition 3.4 of [22]).

The $\mathsf{SampleRight}$ algorithm, on input $(\mathbf{H}_f,\mathbf{R}_f,\mathbf{z})$, exploits this trapdoor to sample a short vector $\tilde{\sigma}_f\in\mathbb{Z}^{3m}$ satisfying $\mathbf{H}_f\cdot\tilde{\sigma}_f=\mathbf{z}$ and $\|\tilde{\sigma}_f\|_\infty\leq\tilde{\beta}_{\max}$. Both conditions are exactly those checked by $\mathsf{HVerify}$, so $\mathsf{HVerify}(vk,\tau,\tilde{\sigma}_f,y,f)=1$. □

A.6 Proof of Theorem 6

Proof. Suppose an adversary $\mathcal{A}$ outputs a forgery $(f^*,y',\tilde{\sigma}')$ s.t. $\mathsf{HVerify}(vk,\tau^*,\tilde{\sigma}',y',f^*)=1$ but $y'\neq y=f^*(\boldsymbol{\mu})$. This means $\mathbf{H}_{f^*}\cdot\tilde{\sigma}'=\mathbf{z}$ where $\mathbf{H}_{f^*}=[\mathbf{A}_\tau\,|\,\mathbf{D}_{f^*}+(y'-1)\mathbf{G}_n]$. The reduction $\mathcal{B}$ also knows the honest evaluated signature $\mathbf{R}_f$ certifying $y=f^*(\boldsymbol{\mu})$. It computes $\tilde{\sigma}\leftarrow\mathsf{Hide}(vk,y,\sigma_f)$, obtaining $\mathbf{H}'_f\cdot\tilde{\sigma}=\mathbf{z}$ where $\mathbf{H}'_f=[\mathbf{A}_\tau\,|\,\mathbf{D}_{f^*}+(y-1)\mathbf{G}_n]$.

Write $\tilde{\sigma}'=(\mathbf{r}'_1,\mathbf{r}'_2)\in\mathbb{Z}^{2m}\times\mathbb{Z}^m$ and $\tilde{\sigma}=(\mathbf{r}_1,\mathbf{r}_2)\in\mathbb{Z}^{2m}\times\mathbb{Z}^m$. Then:

$$\mathbf{A}_\tau\mathbf{r}'_1+(\mathbf{D}_{f^*}+(y'-1)\mathbf{G}_n)\mathbf{r}'_2=\mathbf{z},$$
$$\mathbf{A}_\tau\mathbf{r}_1+(\mathbf{D}_{f^*}+(y-1)\mathbf{G}_n)\mathbf{r}_2=\mathbf{z}.$$

Subtracting and rearranging, we will have:

$$\mathbf{A}_\tau(\mathbf{r}'_1-\mathbf{r}_1)+\mathbf{D}_{f^*}(\mathbf{r}'_2-\mathbf{r}_2)+(y'-1)\mathbf{G}_n\mathbf{r}'_2-(y-1)\mathbf{G}_n\mathbf{r}_2=\mathbf{0}.$$

Since $y'\neq y$, the $\mathbf{G}_n$-components do not cancel. Expanding $\mathbf{A}_\tau=[\mathbf{A}^*\,|\,\mathbf{A}^*\mathbf{U}]$ (in the Type II simulation) and collecting terms involving $\mathbf{A}^*$, one extracts a short non-zero vector $\mathbf{w}$ with $\mathbf{A}^*\mathbf{w}=c\cdot\mathbf{G}_n$ for some $c\neq 0$. Multiplying by $\mathbf{T}_{\mathbf{G}_n}$ yields $\mathbf{A}^*(\mathbf{w}\cdot\mathbf{T}_{\mathbf{G}_n})=\mathbf{0}$. The norm bound $\|\mathbf{w}\cdot\mathbf{T}_{\mathbf{G}_n}\|_\infty\leq m\cdot\tilde{\beta}^2_{\max}\cdot\sqrt{5}\leq\beta_{\mathsf{SIS}}$ holds for appropriately chosen parameters, and the non-zeroness follows from the entropy argument on $\mathbf{r}_0$ (cf. the proof of Theorem 3.1 in [21]): $H_\infty(\mathbf{r}_0\,|\,\mathbf{z}=\mathbf{A}^*\mathbf{r}_0)\geq m-n\log q=\omega(\log\lambda)$, so the probability that the extracted vector is zero is negligible. □

A.7 Proof of Theorem 7

Proof. We need to show that $\mathsf{Hide}(vk,y,\sigma_f)\overset{\text{stat}}{\approx}\mathsf{Sim}(sk,vk,\tau,y,f)$.

The Hide algorithm. Given $\sigma_f=(\mathbf{R}_f,y,f)$ with $\mathbf{A}_\tau\mathbf{R}_f=\mathbf{D}_f+y\mathbf{G}_n$, the Hide procedure forms $\mathbf{H}_f=[\mathbf{A}_\tau\,|\,\mathbf{D}_f+(y-1)\mathbf{G}_n]$ where the second block equals $\mathbf{A}_\tau\mathbf{R}_f+(-1)\mathbf{G}_n$, so $\mathbf{R}_f$ is a $\mathbf{G}_n$-trapdoor for $\mathbf{H}_f$ with tag $h=-1\neq 0$. It outputs $\tilde{\sigma}_f\leftarrow\mathsf{SampleRight}(\mathbf{H}_f,\mathbf{R}_f,\mathbf{z})$.

The Sim algorithm. Given $(sk=\mathbf{T}_{\mathbf{A}^*},vk,\tau,y,f)$, the simulator recomputes $\mathbf{D}_f$ from the public parameters and the circuit f, and forms the same matrix $\mathbf{H}_f=[\mathbf{A}_\tau\,|\,\mathbf{D}_f+(y-1)\mathbf{G}_n]$. Writing $\mathbf{A}_\tau=[\mathbf{A}^*\,|\,\mathbf{A}+\mathbf{B}_\tau]$, we have

$$\mathbf{H}_f=[\mathbf{A}^*\,|\,\mathbf{A}+\mathbf{B}_\tau\,|\,\mathbf{D}_f+(y-1)\mathbf{G}_n].$$

Since $\mathbf{T}_{\mathbf{A}^*}$ is a short basis for $\Lambda^\perp_q(\mathbf{A}^*)$, the simulator invokes $\tilde{\sigma}_f\leftarrow\mathsf{SampleLeft}(\mathbf{A}^*,[\mathbf{A}+\mathbf{B}_\tau\,|\,\mathbf{D}_f+(y-1)\mathbf{G}_n],\mathbf{T}_{\mathbf{A}^*},\mathbf{z},\tilde{s})$, which uses the trapdoor of the *left* block $\mathbf{A}^*$ to sample from the coset $\Lambda^{\mathbf{z}}_q(\mathbf{H}_f)$.

The key observation is that both SampleRight and SampleLeft produce outputs drawn from distributions within negligible statistical distance of the discrete Gaussian $\mathcal{D}_{\Lambda^{\mathbf{z}}_q(\mathbf{H}_f),\tilde{s}}$ over the same coset $\Lambda^{\mathbf{z}}_q(\mathbf{H}_f)=\{\mathbf{v}\in\mathbb{Z}^{3m}:\mathbf{H}_f\cdot\mathbf{v}=\mathbf{z}\bmod q\}$, provided the Gaussian parameter $\tilde{s}$ is sufficiently large. Specifically:

- SampleRight requires $\tilde{s}\geq\|\mathbf{R}_f\|_\infty\cdot p_{\text{extra}}(n,m,\log q)$ and uses the $\mathbf{G}_n$-trapdoor induced by $\mathbf{R}_f$ with tag $h=-1\neq 0$.
- SampleLeft requires $\tilde{s}>\|\widetilde{\mathbf{T}}_{\mathbf{A}^*}\|\cdot\omega(\sqrt{\log 3m})$ and uses the basis $\mathbf{T}_{\mathbf{A}^*}$ of $\Lambda^\perp_q(\mathbf{A}^*)$.

By Lemma 6.4 of [21] (adapted from [88]), for any $(\mathbf{A}^*,\mathsf{td})\in\mathsf{TrapGen}(1^n,1^m,q)$, any $\mathbf{z}\in\mathbb{Z}^n_q$, any $\mathbf{R}_f$ with $\|\mathbf{R}_f\|_\infty\leq\beta_{\max}$, and any $h\neq 0$, we have:

$$\mathsf{SampleLeft}(\mathbf{H}_f,\mathsf{td},\mathbf{z})\overset{\text{stat}}{\approx}\mathsf{SampleRight}(\mathbf{H}_f,\mathbf{R}_f,\mathbf{z}),$$

where the statistical distance is $\mathsf{negl}(n)$. Setting $\tilde{s}=\tilde{\beta}_{\max}=\beta_{\max}\cdot p_{\text{extra}}(n,m,\log q)$ to satisfy both requirements simultaneously, we obtain

$$\mathsf{Hide}(vk,y,\sigma_f)\overset{\text{stat}}{\approx}\mathsf{Sim}(sk,vk,\tau,y,f).$$

Note that Sim depends only on (sk,vk,τ,y,f) and not on the underlying data $\boldsymbol{\mu}$ or the original signature σ_f. Therefore, the hidden signature $\tilde{\sigma}_f$ reveals no information about $\boldsymbol{\mu}$ beyond $y=f(\boldsymbol{\mu})$. Furthermore, the tag component in vk is $\mathbf{A}_\tau=[\mathbf{A}^*\,|\,\mathbf{A}+\mathbf{B}_\tau]$ which depends only on τ and the public parameters, not on $\{\mu_i\}$. □

A.8 Formal Proofs for Phoenix Security

Proof of Theorem 8. We proceed by a sequence of hybrid games $G_0,\ldots,G_4$, where $G_0 := \mathsf{Exp}^{\mathsf{du\text{-}mb\text{-}BPRIV},0}_{\mathcal{A},\mathsf{Phoenix},\mathsf{Sim},\mathsf{Recover}}(\lambda)$ and G_4 is shown indistinguishable from the experiment $\mathsf{Exp}^{\mathsf{du\text{-}mb\text{-}BPRIV},1}_{\mathcal{A},\mathsf{Phoenix},\mathsf{Sim},\mathsf{Recover}}(\lambda)$.

Game G_0 (Real, $\beta=0$). This is the original experiment with $\beta=0$: $\mathcal{O}_{\mathsf{voteLR}}$ returns b_0, $\mathcal{O}_{\mathsf{board}}$ publishes $\mathsf{Publish}(BB_0)$, and $\mathcal{O}_{\mathsf{tally}}$ tallies BB directly.

Game G_1 (Replace FHS signatures with simulated ones). Modify $\mathcal{O}_{\mathsf{voteLR}}$: for each honest ballot $b_0 = (id, upk, c_0, \sigma_0, s_0)$, replace the signature component of $s_0 \leftarrow \mathsf{FHS.Sign}(usk,upk,id,(c_0,\sigma_0),1)$ with $\tilde{s}_0 \leftarrow \mathsf{FHS.Sim}(sk_{fhs},upk,id,1,\mathsf{id}_1)$, i.e. a context-hidden signature produced by the simulator of Definition 10. By the statistical context-hiding property of FHS, the output of Hide and Sim are statistically indistinguishable (cf. the SampleLeft–SampleRight closeness of Lemma 6.4 in [21]), so

$$|\Pr[G_0=1]-\Pr[G_1=1]| \le \mathsf{Adv}^{\mathsf{ch}}_{\mathsf{FHS}}(\lambda).$$

Game G_2 (Replace ZKP proofs with simulated ones). Modify $\mathcal{O}_{\mathsf{voteLR}}$: replace each $\sigma_0 \leftarrow \mathsf{ZKP.Prove}(pp,c_0,v_0)$ with $\sigma_0 \leftarrow \mathsf{ZKP.Sim}(pp,c_0)$, produced by the zero-knowledge simulator without knowledge of v_0. Since ZKP is computationally zero-knowledge under MLWE, transcripts produced by ZKP.Sim are computationally indistinguishable from real proofs. Note that FHS signatures have already been replaced with simulated ones in G_1 and thus do not depend on the content of σ_0. Hence

$$|\Pr[G_1=1]-\Pr[G_2=1]| \le \mathsf{Adv}^{\mathsf{zk}}_{\mathsf{ZKP}}(\lambda).$$

Game G_3 (Switch ciphertexts from v_0 to v_1). Modify $\mathcal{O}_{\mathsf{voteLR}}$ by replacing the $c_0 \leftarrow \mathsf{BFV.Enc}(pk,v_0)$ with $c_1 \leftarrow \mathsf{BFV.Enc}(pk,v_1)$. After G_2, neither σ_0 (simulated, independent of plaintext) nor $\tilde{s}_0$ (context-hidden, independent of signed message content) depend on the encrypted value. We construct a reduction $\mathcal{R}$ to IND-CPA: $\mathcal{R}$ receives a challenge ciphertext $c^* \leftarrow \mathsf{BFV.Enc}(pk,v_b)$ for unknown $b \in \{0,1\}$, embeds it in $\mathcal{O}_{\mathsf{voteLR}}$, and uses $\mathcal{A}$'s output to distinguish. Over q_{Vo} voting queries, a standard hybrid argument yields

$$|\Pr[G_2=1]-\Pr[G_3=1]| \le q_{\mathsf{Vo}} \cdot \mathsf{Adv}^{\mathsf{ind\text{-}cpa}}_{\mathsf{BFV}}(\lambda).$$

Game G_4 (Switch to $\beta = 1$ experiment structure). Modify $\mathcal{O}_{\mathsf{tally}}$: instead of tallying BB directly, compute $BB' \leftarrow \mathsf{Recover}(BB, BB_0, BB_1)$, tally BB', and return $(r,\mathsf{Sim}(pd,\mathsf{Publish}(BB),r))$.

We define Recover as follows. Parse BB and identify each entry originating from an honest voter $id \in \mathcal{H}$ by matching upk; replace the ciphertext c_1 in each such entry with the corresponding c_0 from BB_0 (stored by $\mathcal{O}_{\mathsf{voteLR}}$), and apply BFV.Rerand to the result. For dishonest entries, retain them unmodified. This defines a selection function π as in Definition 11.

Since in G_3 the ciphertexts already encrypt v_1, and Recover substitutes them with encryptions of v_0 followed by re-randomisation, the tallied result r equals $\mathsf{Count} \circ \mathsf{Policy}$ applied to the votes on BB_0 (for honest entries) and the original adversarial entries. By the IND-CPA security of BFV, re-randomised ciphertexts are indistinguishable from fresh encryptions. The simulated tally proof $\mathsf{Sim}(pd,\mathsf{Publish}(BB),r)$ is indistinguishable from a real proof by the zero-knowledge property already applied.

For $\mathcal{O}_{\mathsf{verify}}$: an honest voter $id \in \mathcal{H}_{\mathsf{check}}$ stored st^0_{post} in both G_0 and G_4. The check $H(\tilde{b}) \in pbb$ in the $\beta=1$ world uses $\mathsf{Publish}(BB_1)$, and the voter's stored ballot b_1 is on

BB_1, so VerifyVote succeeds. By collision resistance of H, the adversary cannot cause a false positive or negative:

$$|\Pr[G_3=1]-\Pr[G_4=1]| \leq \mathsf{Adv}^{\mathsf{cr}}_{H}(\lambda)+\mathsf{negl}(\lambda).$$

In G_4 we now undo the changes of G_1–G_3 in the $\beta=1$ world: restore real ZKP proofs and real FHS signatures (now computed on v_1 ciphertexts). Each reversal incurs the same advantage bounds by symmetry. Game G_4 with all reversals applied is exactly $\mathsf{Exp}^{\mathsf{du\text{-}mb\text{-}BPRIV},1}$. By summing over all transitions, we will have:

$$\mathsf{Adv}^{\mathsf{du\text{-}mb\text{-}bpriv}}_{\mathcal{A},\mathsf{Phoenix},\mathsf{Sim}}(\lambda)=|\Pr[G_0=1]-\Pr[\mathsf{Exp}^1=1]|$$
$$\leq 2\mathsf{Adv}^{\mathsf{ch}}_{\mathsf{FHS}}(\lambda)+2\mathsf{Adv}^{\mathsf{zk}}_{\mathsf{ZKP}}(\lambda)+q_{\mathsf{Vo}}\cdot\mathsf{Adv}^{\mathsf{ind\text{-}cpa}}_{\mathsf{BFV}}(\lambda)+\mathsf{Adv}^{\mathsf{cr}}_{H}(\lambda)+\mathsf{negl}(\lambda).$$

□

Proof of Theorem 9. We must show that the three conditions of strong consistency hold for Phoenix with the extraction algorithm $\mathsf{Extract}(b,sk)$ that parses $b=(id,upk,c,\sigma,s)$ and returns $(upk,\mathsf{BFV.Dec}(sk,c))$, and the individual validation algorithm $\mathsf{ValidInd}(b,pk)$ that returns 1 iff $\mathsf{ZKP.Verify}(pp,c,\sigma)=1\wedge\mathsf{FHS.Verify}(upk,id,(c,\sigma),s)=1$.

Condition 1 (Extraction correctness). Let $(pk,sk)\leftarrow\mathsf{Setup}(1^\lambda)$, $(upk,usk)\leftarrow\mathsf{Register}(id)$, and $b\leftarrow\mathsf{Vote}(id,v,pk,usk)$. Then $c=\mathsf{BFV.Enc}(pk,v)$, and by correctness of BFV (Definition 2), $\mathsf{BFV.Dec}(sk,c)=v$ with overwhelming probability provided noise remains below the decryption threshold. The public credential $upk=\mathsf{FHS.KeyGen}(pk_{fhs},id)$ is deterministically recovered. Hence $\mathsf{Extract}(b,sk)=(upk,v)$ with probability $\geq 1-\mathsf{negl}(\lambda)$.

Condition 2 (Valid $\Rightarrow$ ValidInd). $\mathsf{Valid}(BB,b,pk)$ checks $e_1\wedge e_2\wedge e_3$ where $e_2=\mathsf{ZKP.Verify}(pp,c,\sigma)$ and $e_3=\mathsf{FHS.Verify}(upk,id,(c,\sigma),s)$. Since $\mathsf{ValidInd}(b,pk)=e_2\wedge e_3$ and $e_1\wedge e_2\wedge e_3\Rightarrow e_2\wedge e_3$, the implication holds unconditionally.

Condition 3 (Consistency experiment). Let $\mathcal{B}$ output BB such that $\mathsf{ValidInd}(b_i,pk)=1$ for all $b_i\in BB$. Define $dbb[i]=\mathsf{Extract}(b_i,sk)=(upk_i,v_i)$ where $v_i=\mathsf{BFV.Dec}(sk,c_i)$. Set $r'=\mathsf{Count}\circ\mathsf{Policy}(dbb)$.

By knowledge soundness of ZKP under MSIS, since $\mathsf{ZKP.Verify}(pp,c_i,\sigma_i)=1$, there exists an extracted witness v_i' such that $c_i=\mathsf{BFV.Enc}(pk,v_i')$ except with probability $\mathsf{Adv}^{\mathsf{ks}}_{\mathsf{ZKP}}(\lambda)$. Correctness of BFV then gives $\mathsf{BFV.Dec}(sk,c_i)=v_i'$ with overwhelming probability, so $v_i=v_i'$.

Now consider $\mathsf{Tally}(BB,sk)$. After applying Policy, Tally re-checks validity $(e_1\wedge e_2)$ and collects ciphertexts $\{c_j\}_{j\in J}$ for the accepted subset J. Since all ballots pass ValidInd, the checks succeed and J indexes exactly the ballots selected by Policy. The aggregation computes $c^*=\mathsf{BFV.Add}(c_{j_1},\ldots,c_{j_{|J|}})$. By additive homomorphism (Definition 2):

$$r=\mathsf{BFV.Dec}(sk,c^*)=\sum_{j\in J}v_j \pmod t=\mathsf{Count}(\{v_j\}_{j\in J})=\mathsf{Count}\circ\mathsf{Policy}(dbb)=r'.$$

Hence $\Pr[r\neq r']\leq|BB|\cdot\mathsf{Adv}^{\mathsf{ks}}_{\mathsf{ZKP}}(\lambda)+\mathsf{negl}(\lambda)$. □

Proof of Theorem 10. Fix an arbitrary PPT adversary $\mathcal{B}$. Let $(pk,sk)\leftarrow\mathsf{Setup}(1^\lambda)$ and $(upk,usk)\leftarrow\mathsf{Register}(id)$ for each $id\in\mathcal{I}$. Let $(id,v,BB)\leftarrow\mathcal{B}(pk,uL)$ with $id\in\mathcal{I}$ and $b\leftarrow\mathsf{Vote}(id,v,pk,usk)$. Parse $b=(id,upk,c,\sigma,s)$. The experiment returns 1 iff $e_1\wedge\neg\mathsf{Valid}(BB,b,pk)$, i.e. $e_1\wedge\neg(e_1'\wedge e_2\wedge e_3)$ where e_1' is the credential-consistency check evaluated with respect to $BB\cup\{b\}$.

We show $e_2=1$ and $e_3=1$ with overwhelming probability, which forces $\neg(e_1'\wedge e_2\wedge e_3)\Rightarrow\neg e_1'$. But the experiment only returns 1 when e_1 holds, i.e. the adversarial board

BB does not conflict with the honest (id,upk) pair. Under condition e_1, the check e_1' in Valid evaluates identically (since b introduces no new conflict), giving $e_1'=e_1=1$. Hence $e_1\wedge\neg(e_1'\wedge e_2\wedge e_3)=e_1\wedge\neg(1\wedge1\wedge1)=0$. It remains to bound $\Pr[e_2=0]+\Pr[e_3=0]$:

(i) e_2: $\sigma\leftarrow\mathsf{ZKP.Prove}(pp,c,v)$ with $(c,v)\in\mathcal{R}$ since $c=\mathsf{BFV.Enc}(pk,v)$. By completeness of ZKP (Definition 3), $\mathsf{ZKP.Verify}(pp,c,\sigma)=1$ with probability $\geq 1-\mathsf{negl}(\lambda)$.
(ii) e_3: $s\leftarrow\mathsf{FHS.Sign}(usk,upk,id,(c,\sigma),1)$ is honestly generated. By correctness of FHS (the verification equation $\mathbf{A}_\tau\mathbf{R}_f=\mathbf{D}_f+f(\boldsymbol{\mu})\mathbf{G}_n$ holds with $\|\mathbf{R}_f\|_\infty\leq B$ for honestly sampled signatures, as guaranteed by the Gaussian parameter $s\geq\|\widetilde{\mathbf{T}_{\mathbf{A}^*}}\|\cdot\omega(\sqrt{\log m})$), $\mathsf{FHS.Verify}(upk,id,(c,\sigma),s)=1$ with probability $\geq 1-\mathsf{negl}(\lambda)$.

By a union bound, $\mathsf{Adv}^{\mathsf{corr}}_{\mathcal{B},\mathsf{Phoenix},\mathcal{I}}(\lambda)\leq\mathsf{negl}(\lambda)$. □

Proof of Theorem 11. We invoke the composition theorem of Cortier, Galindo, and Warinschi [9], which states: if a voting scheme $\mathcal{V}$ is strongly correct, strongly consistent with extraction algorithm $\mathsf{Extract}$ and counting function Count, and satisfies $\mathsf{du\text{-}mb\text{-}BPRIV}$ with simulator Sim and recovery $\mathsf{Recover}$, then $\mathcal{V}$ securely realises $\mathcal{F}_{\text{voting}}(\rho)$ where $\rho=\mathsf{Count}\circ\mathsf{Policy}$.

By Theorem 10, $\mathsf{Phoenix}$ is strongly correct. By Theorem 9, $\mathsf{Phoenix}$ is strongly consistent with $\mathsf{Extract}(b,sk)=(upk,\mathsf{BFV.Dec}(sk,c))$. By Theorem 8, $\mathsf{Phoenix}$ satisfies $\mathsf{du\text{-}mb\text{-}BPRIV}$. We construct the ideal-world simulator $\mathcal{S}$ as follows. $\mathcal{S}$ internally runs $\mathcal{A}$ and simulates the real-world view:

(i) On $\mathsf{ack}(id)$ from $\mathcal{F}_{\text{voting}}$ (indicating an honest vote was cast), $\mathcal{S}$ creates a simulated ballot $b^*=(id,upk,c^*,\sigma^*,s^*)$ where $c^*\leftarrow\mathsf{BFV.Enc}(pk,0)$, $\sigma^*\leftarrow\mathsf{ZKP.Sim}(pp,c^*)$, and $s^*\leftarrow\mathsf{FHS.Sim}(sk_{fhs},upk,id,1,\mathsf{id}_1)$. By IND-CPA of BFV, c^* is indistinguishable from an encryption of any v; by zero-knowledge of ZKP, σ^* is indistinguishable from a real proof; and by context-hiding of FHS, s^* is indistinguishable from a real signature.
(ii) On receiving the tally result r from $\mathcal{F}_{\text{voting}}$, $\mathcal{S}$ produces a simulated proof $\Pi^*\leftarrow\mathsf{Sim}(pd,\mathsf{Publish}(BB),r)$ and returns (r,Π^*) to $\mathcal{A}$.

Consider the hybrid $H_0=\mathsf{realexec}(\mathfrak{E}\|\mathcal{A}\|\mathsf{Phoenix})$ and $H_1=\mathsf{idealexec}(\mathfrak{E}\|\mathcal{S}\|\mathcal{F}_{\text{voting}}(\rho))$. The transition $H_0\to H_1$ decomposes as:

$$H_0\xrightarrow{\mathsf{FHS}\text{ ctx-hiding}}H_{0.1}\xrightarrow{\mathsf{ZKP}\text{ zero-knowledge}}H_{0.2}\xrightarrow{\mathsf{BFV}\text{ IND-CPA}}H_{0.3}\xrightarrow{\text{tally simulation}}H_1,$$

where each step incurs advantage bounded by the corresponding primitive's security. The tally simulation step is exact: by strong consistency, the real tally equals $\rho(L)$ where L is the list of extracted votes, which is precisely the value returned by $\mathcal{F}_{\text{voting}}$. Thus

$$\big|\Pr[H_0=1]-\Pr[H_1=1]\big|\leq\mathsf{Adv}^{\mathsf{du\text{-}mb\text{-}bpriv}}_{\mathcal{A},\mathsf{Phoenix},\mathsf{Sim}}(\lambda)+\mathsf{negl}(\lambda).$$ □

Proof of Theorem 12. We prove that $\Pr[\mathsf{Exp}^{\mathsf{ver\text{-}b}}_{\mathcal{A},\mathsf{Phoenix}}(\lambda)=1]$ and $\Pr[\mathsf{Exp}^{\mathsf{ver\text{-}r}}_{\mathcal{A},\mathsf{Phoenix}}(\lambda)=1]$ are negligible.

Experiment $\mathsf{Exp}^{\mathsf{ver\text{-}b}}$ (honest registrar, adversarial bulletin board). Suppose $\mathcal{A}$ outputs (BB,r,Π) with $\mathsf{Verify}(BB,r,\Pi)=1$. By knowledge soundness of ZKP applied to the tally proof Π, there exists an extractor $\mathcal{K}$ that, with overwhelming probability, recovers sk such that $r=\mathsf{BFV.Dec}(sk,c^*)$ where $c^*=\mathsf{BFV.Add}(\{c_i\}_{i\in pbb})$, except with probability $\mathsf{Adv}^{\mathsf{ks}}_{\mathsf{ZKP}}(\lambda)$. We partition the ballots on BB into three disjoint sets:

(a) $E=\{b_i:id_i\in Checked\}$: ballots from checked honest voters.
(b) $A=\{b_i:id_i\in\mathcal{H}\setminus Checked\}$: ballots from honest voters who did not verify.

(c) $B = \{b_i : id_i \notin \mathcal{H}\}$: ballots attributed to corrupted or fabricated identities.

Individual verifiability (set E): For each $id \in \mathcal{H}_{\mathsf{check}}$ with honest ballot $b = (id, upk, c, \sigma, s)$, the voter checks $H(\tilde{b}) \in pbb$. If $\mathcal{A}$ substitutes b with $b' \neq b$ while preserving $H(\tilde{b}) = H(\tilde{b'})$, this constitutes a collision in H, occurring with probability $\leq \mathsf{Adv}^{\mathsf{cr}}_H(\lambda)$. If $\mathcal{A}$ omits b entirely, then $\mathsf{CheckedVoter}(id, v, b) = 0$, so $id \notin \mathit{Checked}$ and the experiment returns 0 (the winning condition requires checked voters' ballots to be present). Hence, for each $id \in \mathit{Checked}$, the ballot on BB decrypts to v_i^E except with probability $\mathsf{Adv}^{\mathsf{cr}}_H(\lambda)$.

Eligibility verifiability (set B): Each ballot $b_i \in B$ must satisfy the value of the $\mathsf{FHS.Verify}(upk_i, id_i, (c_i, \sigma_i), s_i) = 1$ to be included. Since the registrar is honest, usk for $id \notin CU$ was never revealed to $\mathcal{A}$. Suppose $|B| > |CU|$; then at least one $b_j \in B$ bears id_j for which $\mathcal{A}$ does not hold usk_j. A valid signature s_j under upk_j for a message not signed by the honest signer constitutes an existential forgery against FHS. By a standard reduction: if $|B| > |CU|$ with non-negligible probability, we construct a forger $\mathcal{F}$ against FHS that succeeds with probability $\geq \frac{1}{|\mathcal{I}|} \cdot \Pr[|B| > |CU|]$, which yields $\Pr[|B| > |CU|] \leq |\mathcal{I}| \cdot \mathsf{Adv}^{\mathsf{euf}}_{\mathsf{FHS}}(\lambda)$.

Universal verifiability (global consistency): By correctness of BFV and additive homomorphism, $r = \sum_{i \in BB} v_i = \sum_{i \in E} v_i^E + \sum_{i \in A} v_i^A + \sum_{i \in B} v_i^B$ where each v_i is the plaintext extracted via $\mathsf{BFV.Dec}$. Since $|B| \leq |CU|$ and votes in E are fixed by the individual verifiability argument, the tally decomposes as $r = \rho(\{v_i^E\}) \oplus_R \rho(\{v_i^A\}) \oplus_R \rho(\{v_i^B\})$ with $|\{v_i^B\}| \leq |CU|$. Hence the winning condition of $\mathsf{Exp}^{\mathsf{ver\text{-}b}}$ (that no such decomposition exists) fails, and

$$\Pr[\mathsf{Exp}^{\mathsf{ver\text{-}b}}_{\mathcal{A},\mathsf{Phoenix}}(\lambda) = 1] \leq \mathsf{Adv}^{\mathsf{euf}}_{\mathsf{FHS}}(\lambda) \cdot |\mathcal{I}| + |\mathcal{H}| \cdot \mathsf{Adv}^{\mathsf{cr}}_H(\lambda) + \mathsf{Adv}^{\mathsf{ks}}_{\mathsf{ZKP}}(\lambda) + \mathsf{negl}(\lambda).$$

Experiment $\mathsf{Exp}^{\mathsf{ver\text{-}r}}$ (honest bulletin board, adversarial registrar). Here $\mathcal{A}$ controls the registrar and may issue arbitrary credentials, but can only add ballots to BB via $\mathcal{O}_{\mathsf{cast}}(id, b)$, which runs $\mathsf{Box}(BB, b)$ on an honest board that enforces Valid. Each call to $\mathcal{O}_{\mathsf{cast}}$ applies $\mathsf{Valid}(BB, b, pk)$, which verifies $\mathsf{ZKP.Verify}(pp, c, \sigma) = 1$ and $\mathsf{FHS.Verify}(upk, id, (c, \sigma), s) = 1$.

Since $\mathcal{A}$ controls the registrar, it can generate valid credentials for arbitrary identities. However, for honest voters queried via $\mathcal{O}_{\mathsf{vote}}(id, v)$, the credential was generated by $\mathcal{O}_{\mathsf{reg}}(id)$ and usk is unknown to $\mathcal{A}$ unless $\mathcal{O}_{\mathsf{corrupt}}(id)$ is called. The honest bulletin board records all ballots faithfully, so individual verifiability holds trivially: a checked voter's ballot is present on BB by the honest-board guarantee, and its hash appears on pbb since $\mathsf{Publish}$ is applied honestly.

For eligibility, the honest board accepts at most one ballot per identity (via the credential-consistency check e_1 in Valid), and the adversary can inject ballots only for identities in CU for which it holds valid credentials. The argument then follows identically to the $\mathsf{Exp}^{\mathsf{ver\text{-}b}}$ case, giving

$$\Pr[\mathsf{Exp}^{\mathsf{ver\text{-}r}}_{\mathcal{A},\mathsf{Phoenix}}(\lambda) = 1] \leq \mathsf{Adv}^{\mathsf{ks}}_{\mathsf{ZKP}}(\lambda) + \mathsf{negl}(\lambda). \qquad \square$$

References

1. M. Bernhard et al. Public evidence from secret ballots. In *Electronic Voting*, LNCS, pages 84–109. Springer, 2017.
2. J. Budurushi et al. An investigation into the usability of electronic voting systems for complex elections. *Ann. Telecommun.*, 71(7–8):309–322, 2016.

3. D. Rausch, N. Huber, and R. Küsters. Verifiable e-voting with a trustless bulletin board. In *IEEE CSF*, pages 457–472, 2025.
4. G. Kaim et al. Post-quantum online voting scheme. In *FC Workshops*, pages 290–305. Springer, 2021.
5. J. Mascelli and M. Rodden. "harvest now decrypt later": Examining post-quantum cryptography and the data privacy risks for distributed ledger networks. Technical Report FEDS 2025-093, Board of Governors of the Federal Reserve System, 2025.
6. D.J. Bernstein and T. Lange. Post-quantum cryptography. *Nature*, 549(7671):188–194, 2017.
7. D. Sikeridis, P. Kampanakis, and M. Devetsikiotis. Post-quantum authentication in TLS 1.3: A performance study. In *NDSS*, 2020.
8. A. Aydeger et al. Post-quantum public key infrastructures: Hybrid certificates, cryptographic combiners, and migration strategies. In *IEEE CCNC*, pages 1–4, 2026.
9. D. Bernhard et al. Sok: A comprehensive analysis of game-based ballot privacy definitions. In *IEEE S&P*, pages 499–516, 2015.
10. X. Boyen, T. Haines, and J. Müller. Epoque: Practical end-to-end verifiable post-quantum-secure e-voting. In *IEEE EuroS&P*, pages 272–291, 2021.
11. P. Hough, C. Sandsbråten, and T. Silde. More efficient lattice-based electronic voting from NTRU. *IACR Commun. Cryptol.*, 1(4), 2025.
12. R. Mosaheb. *Quantum-Safe Electronic Voting Schemes*. PhD thesis, Université du Luxembourg, 2025.
13. R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
14. I. Black et al. Practical quantum-safe voting from lattices, extended. Cryptology ePrint Archive, Paper 2022/1686, 2022.
15. I. Chillotti et al. A homomorphic LWE based e-voting scheme. In *PQCrypto*, pages 245–265. Springer, 2016.
16. N. Abapour et al. Blockchain-based economic voting with posterior security from lattices. In *IWSEC*, pages 247–266. Springer, 2025.
17. P. Sekar et al. Quantum-resistant electronic voting system using lattice cryptography techniques. In *Internet and Modern Society*, pages 257–270. Springer, 2025.
18. S.P. Mahanayak, B. Nikhita, and S. Bilgaiyan. Enhancing e-voting security with quantum-resistant encryption. *SN Comput. Sci.*, 4(5), 2023.
19. A. Junior Gabriel et al. Post-quantum crystography system for secure electronic voting. *Open Comput. Sci.*, 9(1):292–298, 2019.
20. F. Hao et al. Experience from UNITA elections: Reconciling revote, E2E verifiability and low coercion. Cryptology ePrint Archive, Paper 2025/1669, 2025.
21. S. Gorbunov, V. Vaikuntanathan, and D. Wichs. Leveled fully homomorphic signatures from standard lattices. In *ACM STOC*, pages 469–477, 2015.
22. F. Luo et al. A more efficient leveled strongly-unforgeable fully homomorphic signature scheme. *Inf. Sci.*, 480:70–89, 2019.
23. C.C. Drăgan et al. Machine-checked proofs of privacy against malicious boards for Selene & co. In *IEEE CSF*, pages 335–347, 2022.
24. V. Cortier et al. Sok: Verifiability notions for e-voting protocols. In *IEEE S&P*, pages 779–798, 2016.
25. G. Barthe et al. Easycrypt: A tutorial. In *Foundations of Security Analysis and Design VII*, pages 146–166. Springer, 2014.
26. Anonymous github. https://anonymous.4open.science/r/phoenix1. Accessed: 2026-03-25.
27. R. Küsters, T. Truderung, and A. Vogt. Verifiability, privacy, and coercion-resistance: New insights from a case study. In *IEEE S&P*, pages 538–553, 2011.
28. J. Benaloh. Verifiable secret-ballot elections. In *Ph.D. Thesis, Yale University*, 1987.

29. J. Benaloh et al. ElectionGuard: A cryptographic toolkit to enable verifiable elections. In *USENIX Security*, pages 5485–5502, 2024.
30. X. Boyen, T. Haines, and J. Mueller. Epoque: Practical end-to-end verifiable post-quantum-secure e-voting. Cryptology ePrint Archive, Paper 2021/304, 2021.
31. R. Küsters, T. Truderung, and A. Vogt. Accountability: Definition and relationship to verifiability. In *ACM CCS*, pages 526–535, 2010.
32. D.F. Aranha et al. Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. In *ACM CCS*, pages 1467–1481, 2023.
33. D.F. Aranha et al. Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. Cryptology ePrint Archive, Paper 2022/422, 2022.
34. S. Bouaziz-Ermann et al. Lattice-based (partially) blind signature without restart. Cryptology ePrint Archive, Report 2020/260, 2020.
35. D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *EUROCRYPT*, volume 7237 of *LNCS*, pages 700–718. Springer, 2012.
36. R. Bendlin, S. Krehbiel, and C. Peikert. How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE. In *ACNS*, volume 7954 of *LNCS*, pages 218–236. Springer, 2013.
37. C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *ACM STOC*, pages 197–206, 2008.
38. V. Lyubashevsky, C. Peikert, and O. Regev. A toolkit for ring-LWE cryptography. In *EUROCRYPT*, volume 7881 of *LNCS*, pages 35–54. Springer, 2013.
39. E. Hauck et al. Lattice-based blind signatures, revisited. In *CRYPTO*, volume 12171 of *LNCS*. Springer, 2020.
40. A. Esser, A. May, and F. Zweydinger. McEliece needs a break – solving McEliece-1284 and quasi-cyclic-2918 with modern ISD. In *EUROCRYPT*, volume 13277 of *LNCS*, pages 433–457. Springer, 2022.
41. S. Park et al. Going from bad to worse: From internet voting to blockchain voting. *J. Cybersecur.*, 7(1), 2021.
42. S. Agrawal, D. Boneh, and X. Boyen. Efficient lattice (H)IBE in the standard model. In *EUROCRYPT*, pages 553–572, 2010.
43. R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
44. R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
45. D. Stehlé and R. Steinfeld. Making NTRU as secure as worst-case problems over ideal lattices. In *EUROCRYPT*, volume 6632 of *LNCS*, pages 27–47. Springer, 2011.
46. C. Baum et al. More efficient commitments from structured lattice assumptions. In *SCN*, volume 11035 of *LNCS*, pages 368–385. Springer, 2018.
47. D.F. Aranha et al. Lattice-based proof of shuffle and applications to electronic voting. In *CT-RSA*, volume 12704 of *LNCS*, pages 227–251. Springer, 2021.
48. D.F. Aranha et al. Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. In *ACM CCS*, pages 1467–1481, 2023.
49. J. Bootle et al. More efficient amortization of exact zero-knowledge proofs for LWE. In *ESORICS*, volume 12973 of *LNCS*, pages 608–627. Springer, 2021.
50. E. Bellini et al. Enhancing code based zero-knowledge proofs using rank metric. In *CANS*, volume 12579, pages 570–592, 2020.
51. N. Aragon et al. BIKE: Bit-flipping key encapsulation. https://bikesuite.org/files/v5.0/BIKE_Spec.2022.10.10.1.pdf, 2022.
52. C. Aguilar-Melchor et al. Efficient encryption from random quasi-cyclic codes. *IEEE Trans. Inf. Theory*, 64(5):3927–3943, 2018.
53. F. Benhamouda et al. Better zero-knowledge proofs for lattice encryption and their application to group signatures. In *ASIACRYPT*, pages 551–572, 2014.
54. V. Lyubashevsky. Lattice signatures without trapdoors. In *EUROCRYPT*, pages 738–755, 2012.

55. C. Baum et al. Efficient commitments and zero-knowledge protocols from ring-SIS with applications to lattice-based threshold cryptosystems. Cryptology ePrint Archive 2016/997, 2016.
56. R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
57. L. Ducas and D. Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In *EUROCRYPT*, volume 9056 of *LNCS*, pages 617–640. Springer, 2015.
58. L. Ducas et al. Lattice signatures and bimodal gaussians. In *CRYPTO*, volume 8042 of *LNCS*, pages 40–56. Springer, 2013.
59. F. Laguillaumie et al. Lattice-based group signatures with logarithmic signature size. In *ASIACRYPT*, volume 8270 of *LNCS*, pages 41–61. Springer, 2013.
60. I. Hwang, J. Seo, and Y. Song. Concretely efficient lattice-based polynomial commitment from standard assumptions. In *CRYPTO*, pages 414–448. Springer, 2024.
61. T.H. Yuen et al. Posterior security: Anonymity and message hiding of standard signatures. In *ACM CCS*, pages 528–542, 2025.
62. F. Baldimtsi et al. Non-interactive blind signatures: Post-quantum and stronger security. In *ASIACRYPT*, volume 15485 of *LNCS*. Springer, 2025.
63. J. Bos et al. CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM. Cryptology ePrint Archive, Report 2017/634, 2017.
64. J. Huang et al. The application of the blockchain technology in voting systems. *ACM Comput. Surv.*, 54:1–28, 2021.
65. S. Vivek et al. E-voting systems using blockchain: An exploratory literature survey. In *IEEE ICIRCA*, 2020.
66. National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS PUB 202, NIST, 2015.
67. W. Diffie and M.E. Hellman. New directions in cryptography. *IEEE Trans. Inf. Theory*, 22(6):644–654, 1976.
68. B. Guidi, A. Michienzi, and L. Ricci. Data persistence in decentralized social applications: the IPFS approach. In *IEEE CCNC*, 2021.
69. Y. Lin and C. Zhang. A method for protecting private data in IPFS. In *IEEE CSCWD*, 2021.
70. A.J. Gabriel et al. Post-quantum crystography: A combination of post-quantum cryptography and steganography. In *ICITST*, pages 454–457, 2013.
71. A. Petzoldt. *Selecting and Reducing Key Sizes for Multivariate Cryptography*. PhD thesis, Technical University Darmstadt, 2013.
72. Z. Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In *CRYPTO*, pages 868–886. Springer, 2012.
73. V. Lyubashevsky, N.K. Nguyen, and M. Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In *CRYPTO*, pages 71–101. Springer, 2022.
74. V. Cortier, J. Lallemand, and B. Warinschi. Fifty shades of ballot privacy: Privacy against a malicious board. In *IEEE CSF*, pages 17–32, 2020.
75. V. Cortier et al. Election verifiability for Helios under weaker trust assumptions. In *ESORICS*, pages 327–344. Springer, 2014.
76. M. Ajtai. Generating hard instances of lattice problems (extended abstract). In *ACM STOC*, pages 99–108, 1996.
77. A. Langlois and D. Stehle. Worst-case to average-case reductions for module lattices. *Des. Codes Cryptogr.*, 75(3):565–599, 2015.
78. O. Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), 2009.
79. Z. Brakerski, C. Gentry, and V. Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Trans. Comput. Theory*, 6(3), 2014.
80. National Institute of Standards and Technology. Secure hash standard (SHS). Technical Report FIPS PUB 180-4, NIST, 2015.

81. J. Fan and F. Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012.
82. V. Lyubashevsky, C. Peikert, and O. Regev. On ideal lattices and learning with errors over rings. In *EUROCRYPT*, volume 6110 of *LNCS*, pages 1–23. Springer, 2010.
83. C. Peikert and S. Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In *CRYPTO*, pages 89–114. Springer, 2019.
84. A. Chailloux et al. Interactive and noninteractive zero knowledge are equivalent in the help model. In *TCC*, volume 4948 of *LNCS*. Springer, 2008.
85. J. Groth, R. Ostrovsky, and A. Sahai. Perfect non-interactive zero knowledge for NP. In *EUROCRYPT*, volume 4004 of *LNCS*. Springer, 2006.
86. R. Gay and B. Ursu. On instantiating unleveled fully-homomorphic signatures from falsifiable assumptions. In *PKC*, volume 14601 of *LNCS*. Springer, 2024.
87. C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *ACM STOC*, pages 197–206, 2008.
88. D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *EUROCRYPT*, volume 7237 of *LNCS*, pages 700–718. Springer, 2012.
89. D. Boneh et al. Threshold cryptosystems from threshold fully homomorphic encryption. In *CRYPTO*, pages 565–596. Springer, 2018.
90. D.J. Bernstein. Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? Workshop Record of SHARCS'09, 2009.
91. M. Albrecht et al. Homomorphic encryption security standard. HomomorphicEncryption.org, 2018.
92. M.R. Albrecht, R. Player, and S. Scott. On the concrete hardness of learning with errors. J. Math. Cryptol., 9(3):169–203, 2015.
93. V. Cortier et al. Machine-checked proofs for electronic voting: Privacy and verifiability for Belenios. In *IEEE CSF*, pages 298–312, 2018.
94. S. Garg et al. Threshold encryption with silent setup. In *CRYPTO*, pages 352–386. Springer, 2024.
95. B. Waters and D.J. Wu. Silent threshold cryptography from pairings: Expressive policies in the plain model. Cryptology ePrint Archive, Paper 2025/1547, 2025.
96. M. Hall-Andersen, M. Simkin, and B. Wagner. Silent threshold encryption with one-shot adaptive security. Cryptology ePrint Archive, Paper 2025/1384, 2025.
97. J. Bormet et al. BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Paper 2025/1419, 2025.
98. P. Chaidos et al. BeleniosRF: A non-interactive receipt-free electronic voting scheme. In *ACM CCS*, pages 1614–1625, 2016.
99. S. Delaune, S. Kremer, and M. Ryan. Coercion-resistance and receipt-freeness in electronic voting. In *IEEE CSFW*, pages 28–42, 2006.